

CH9、Advance Tree

高等樹

目錄：

Min-Max Heap(Insert, Delete)

Deap(Insert, Delete)

SMMH(Insert, Delete)

Extended BT

Weight External Path Length

Minimum Weight External Path Length

Huffman

如何評估 BST 優劣

AVL Tree

不平衡的調整方式

定理

m-way Search Tree

B-Tree

2-3 Tree

2-3-4 Tree(Insert, Delete)

B⁺ Tree

Red & Black Tree

OBST

Splay Tree

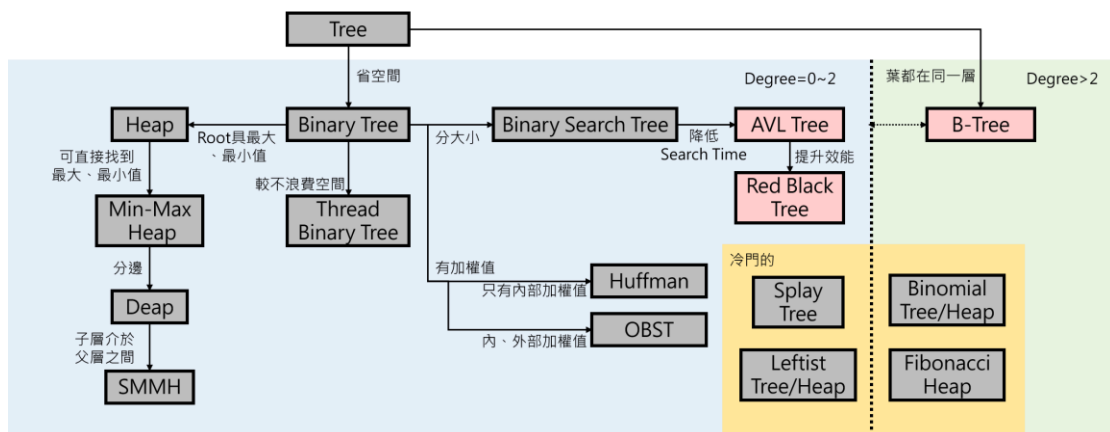
Leftist Tree

Min-Leftist Tree, Max-Leftist Tree(Insert, Delete)

Binomial Tree

Binomial Heap(Insert, Delete)

Fibonacci Heap

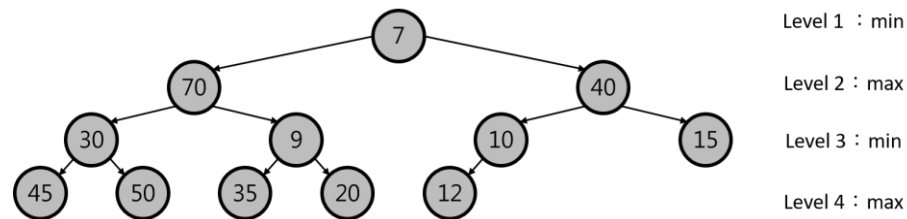


Min-Max Heap(最小最大堆積)

Def: 是一 Complete BT, 滿足:

1. 此種的交替階層為 min-level、max-level
2. Root 位於 min-level
3. 若 x 值於 min-level 表示以 x 的 Root 之子樹中, x 為最小
4. 若 x 值於 max-level 表示以 x 的 Root 之子樹中, x 為最大

例:



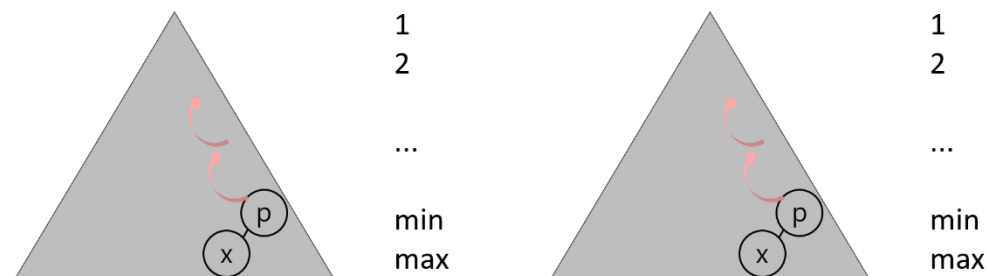
用途: 製作 Double-End Priority Queue 之 Data Structure

提供下列運作:

Operation:

1. $O(\log n)$: Insert、Delete Max、Delete Min
2. $O(1)$: find Min/Max

Insert x 到 Min-Max Heap 中



步驟

1. x 置於 last Node 之下一個位置($n=n+1$)
2. 令 p 為 x 的父點, 分成下死 2 個 case:

(1) p 位於 min-level

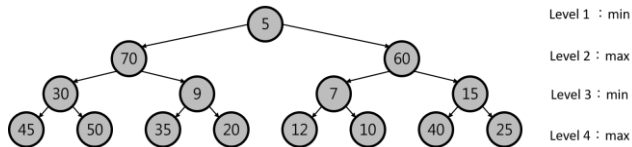
```
if(x.key < H[p].key)
{
    H[n] = H[p];
    VerifyMin(H, p, x);
}
else
    VerifyMax(H, p, x);
```

(2) P 位於 max-level

```

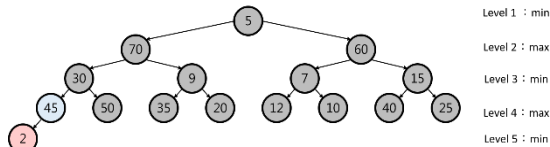
if(x.key < H[p].key)
{
    H[n] = H[p];
    VerifyMax(H, p, x);
}
else
    VerifyMin(H, p, x);
    
```

例：insert：2, 80

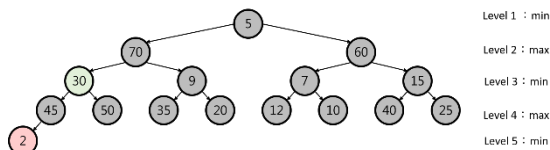


第一題：

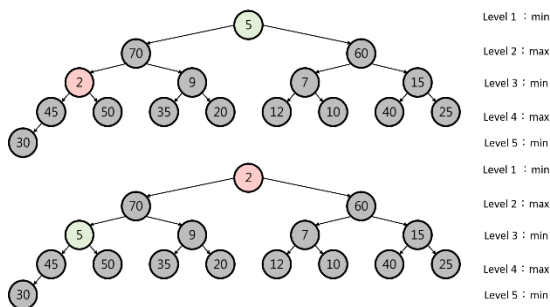
- 依 Complete 方式加入 2、階檢查其父點 45，因為父點位於 max level，且 $45 > 2$ ，故無需交換



- 檢查 2 的祖父點 30，因為皆位於 min level，且 $30 < 2$ ，故交換

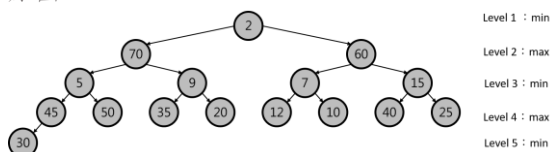


- 檢查 2 的祖父點 5，因為皆位於 min level，且 $2 < 5$ ，故交換(完成)

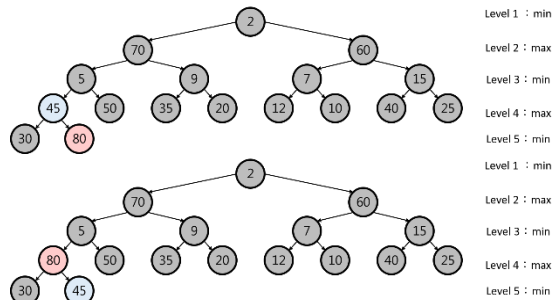


第二題：

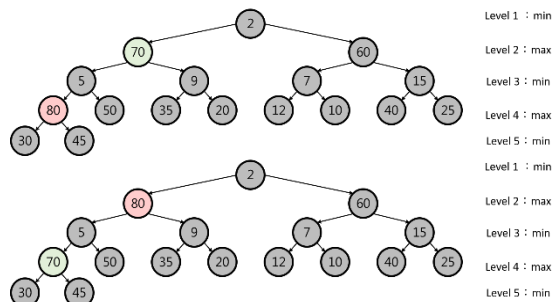
- 原圖：



2. 依 Complete 方式加入 80、階檢查其父點 45，因為父點位於 max level，且 $80 > 45$ ，故交換



3. 檢查 2 的祖父點 5，因為皆位於 max level，且 $80 > 70$ ，故交換(完成)



程式：

```

Procedure MinMaxInsert(h: minmaxheap; n: integer; x: element)
begin
    if n=MaxSize then Full;
    else begin
        n=n+1;
        p=n/2;
        if(p==0) then h[1]=x;
        else case level(p) of
            begin
                case: "min" case1 algo;
                case: "max" case2 algo;
            end
        end
    end
end
end

```

VerifyMax 演算法：

```

Procedure VerifyMax(h, i, x)
var gp: integer;
begin
    gp=i/4;
    while gp<>0 do begin
        if(x.key>h[gp].key)
        begin
            h[i]=h[gp];
            i=gp;
            gp=gp/4;
        end
        else
            gp==0;    h[i]=x;
    end
end
end

```

VerifyMin 演算法：

```

Procedure VerifyMin(h, i, x)
  var gp: integer;
  begin
    gp=i/4;
    while gp<>0 do begin      //<>即是 ≠ or !=
      if(x.key<h[gp].key)
        begin
          h[i]=h[gp];
          i=gp;
          gp=gp/4;
        end
      else
        gp==0;    h[i]=x;
      end
    end
  end
end

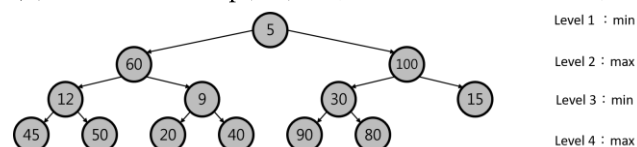
```

Min-Max Heap 之 Delete Min(Root)

步驟：

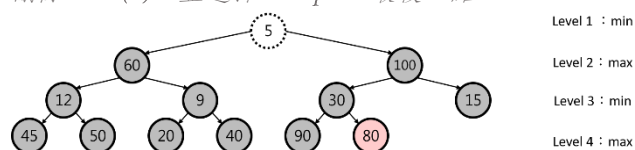
1. 移走 Root
2. 將 last Node x Delete
3. 將 x 插入 Root 中，處理方式分為：
 - (1) Root 無中點，x 於 Root，停止
 - (2) Root 有子點，無子孫，令 k 為子點中最小值：
 - if(k<x)則 swap(x, k)
 - else 則 x 置於 Root
 - (3) Root 有子孫，令 k 為子孫中最小值，令 p 為 k 的父點，則：
 - if(k<x)，則：
 - a. k 置於 Root
 - b. if(x>p)則 swap(x, p)
 - c. 以原本 k 的位置為 Root，goto 3.
 - else 則 x 置於 Root 且停止

例：Min-Max Heap 如下，若 Delete Min 2 次，結果為何？

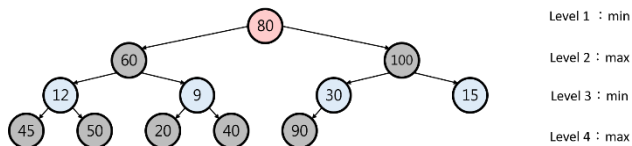


第一次刪除：

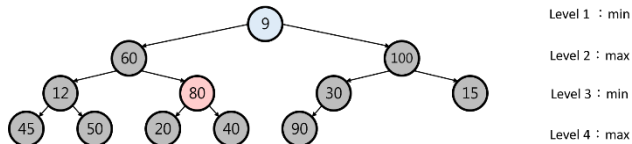
1. 刪除 Root(5)，並選擇 Complete 最後一點 80



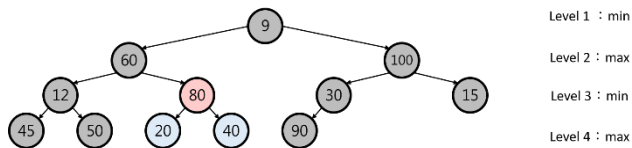
2. 將 80 置於 Root，並且將其子孫選擇出來最小者 9



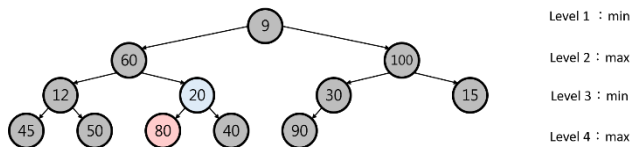
3. Root 80 與其最小子孫 9 比較，因為位於 min level，且 $80 < 9$ ，故交換



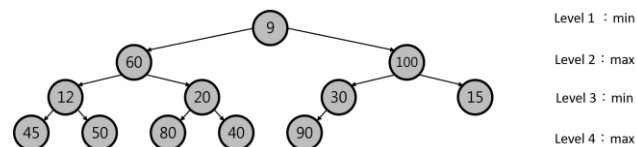
4. 將 8 子孫選擇出來最小者 20



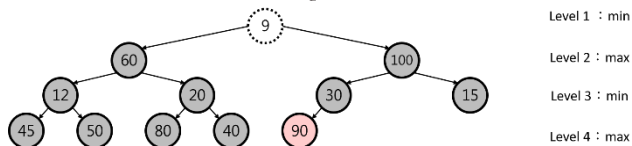
5. 80 與其最小子點 20 比較，因為位於 min level，且 $80 > 20$ ，故交換(完成)



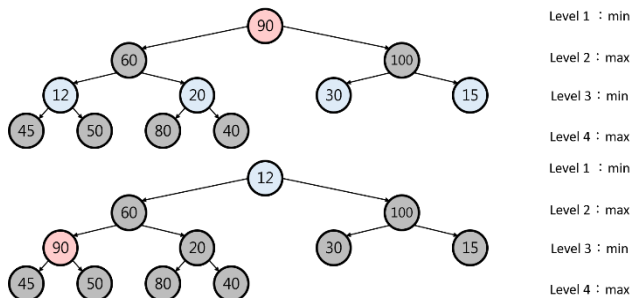
第二次刪除：
原圖：



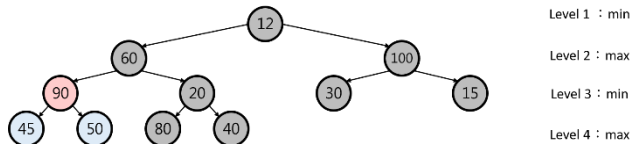
1. 刪除 Root(9)，並選擇 Complete 最後一點 90



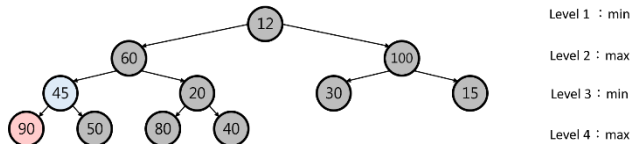
2. Root 90 與其最小子孫 12 比較，因為位於 min level，且 $90 > 12$ ，故交換



3. 將 90 子點選擇出來最小者 45



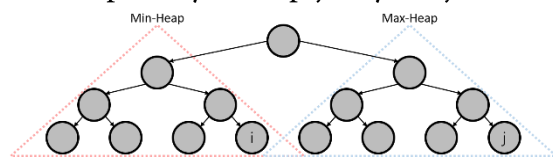
4. 90 與其最小子點 45 比較，因為位於 min level，且 $90 > 45$ ，故交換(完成)



Deap(Double Ended Heap，雙堆積)

Def：為一 Complete BT，滿足：

1. Root 不存 Data
2. 存子樹為 Min-Heap
3. 右子樹為 Max-Heap
4. 若 i 為左子樹之 Node 編號， j 為 i 在右子樹之對應編號，則 $Deap[i].key \leq Deap[j].key$ ，if $j > N$ then $j = [i/2]$



說明：

$j = i + (\text{Node } i \text{ 的上一層之最多 Node 數}) = i + 2^{i \text{ 的 level}-1} = i + 2^{i \text{ 的 level}-2}$ ，又 i 所處的 level 為： $2^k - 1 = i \Rightarrow 2^k = i + 1 \Rightarrow k = \log(i + 1)$

用途：

製作 Double-Ended Priority Queue

提供 Insert、Delete Min、Delete Max 為 $O(\log n)$

Note：就效能而言：Deap 快於 Min-Max Heap

Deap 常見的運作

Insert

步驟：

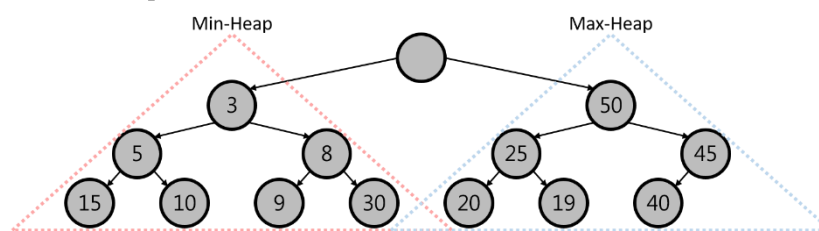
1. x 置於 last Node 的下個位置
2. 分為 2 case 討論
 - (1) x 位於 Min-Heap，令 i 在 Max-Heap 之對應為 j，則：

```
if(x.key>Deap[i].key)
{
    Deap[n]=Deap[j];
    MaxDeapInsert(D, j, x);
}
else
    MinDeapInsert(D, n, x);
```

- (2) x 位於 Max-Heap，令 i 在 Min-Heap 之對應為 j，則：

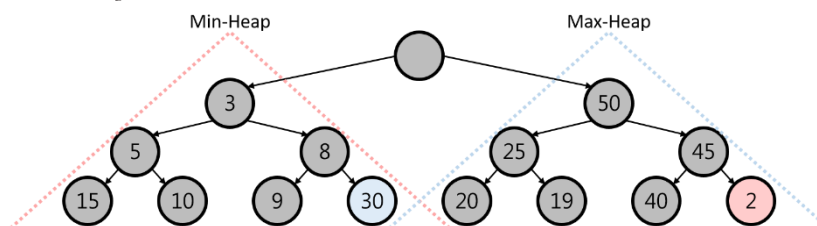
```
if(x.key<Deap[i].key)
{
    Deap[n]=Deap[i];
    MinDeapInsert(D, i, x);
}
else
    MaxDeapInsert(D, n, x);
```

例：一 Deap 如下，insert 2, 60 之結果為何？

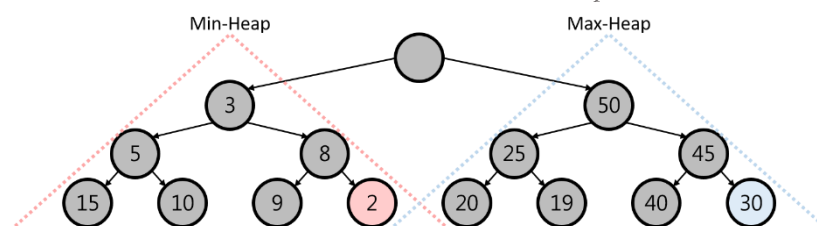


第一次 insert：

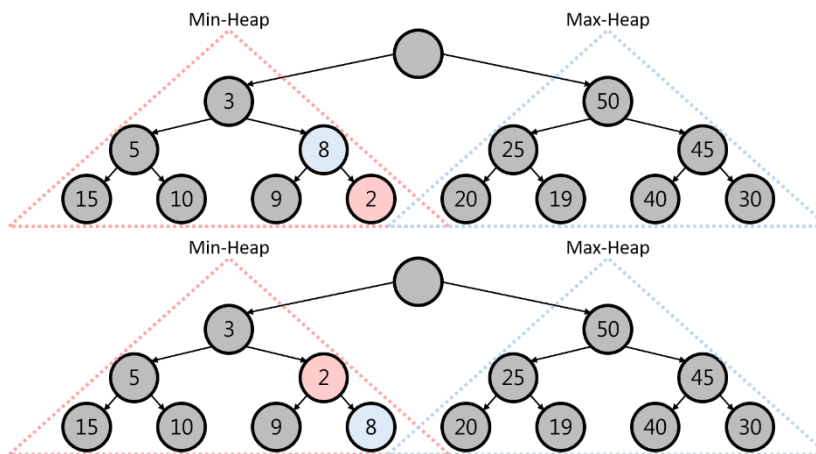
1. 先依 Complete 順序，將 2 放入，且選擇其對應位置點 30



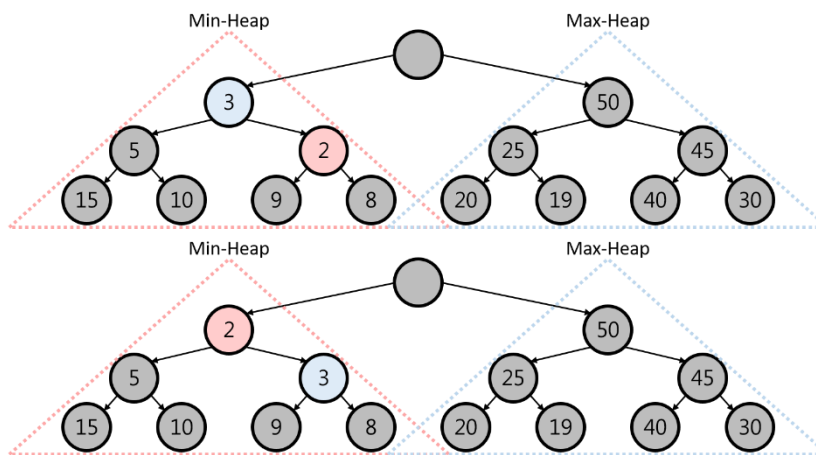
2. 比較 2 點，因為 2<30，故交換，將 2 置於 Min-Heap 的對應位置



3. 檢查 2 之父點，比較大小，因 $2 < 8$ ，且位於 Min-Heap，故交換

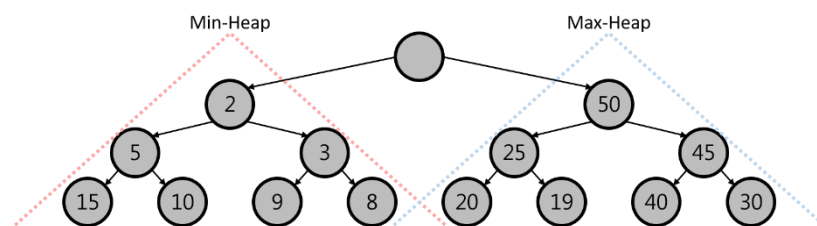


4. 檢查 2 之父點，比較大小，因 $2 < 3$ ，且位於 Min-Heap，故交換(完成)

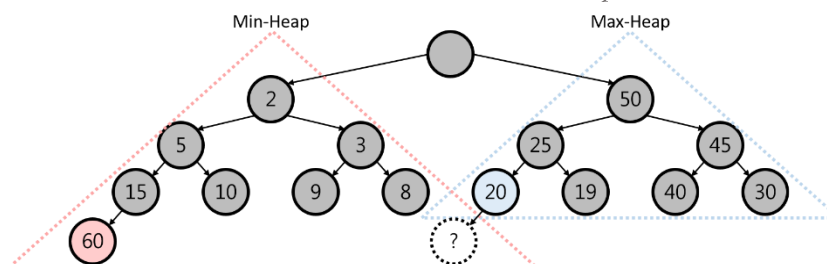


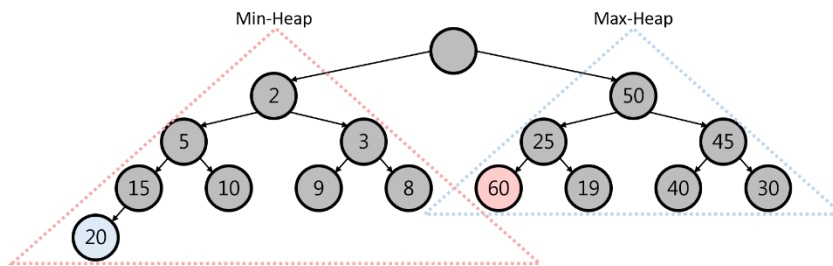
第二次 insert :

原圖：

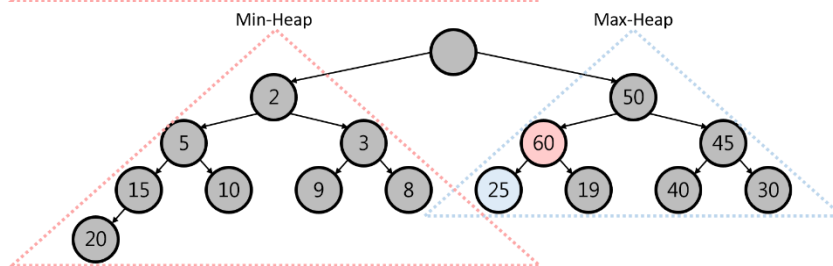
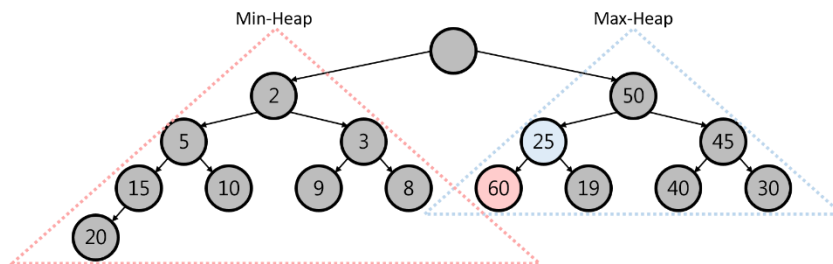


1. 先依 Complete 順序，將 60 放入，且選擇其對應位置點，但因其對應點為空，故取其父點 20 比較；因為 $60 > 20$ ，故交換，將 60 移至 Max-Heap

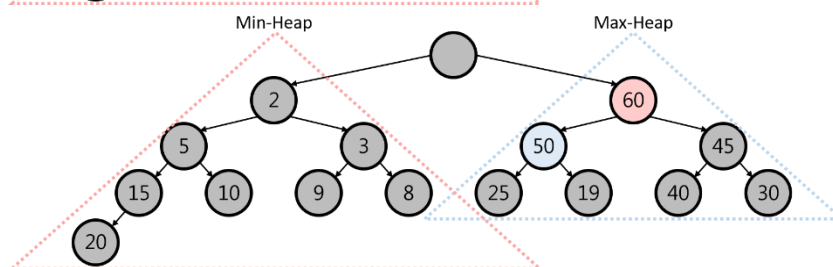
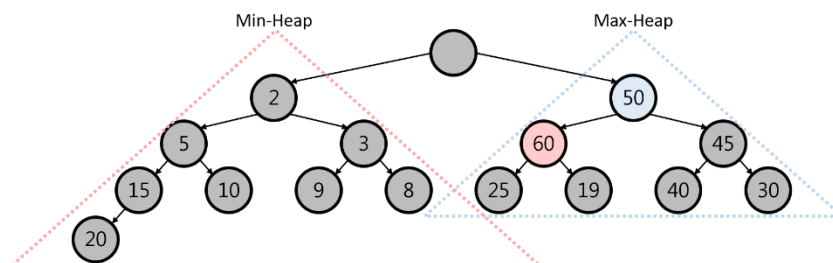




2. 檢查 60 之父點 25，比較大小，因 $60 > 25$ ，且位於 Max-Heap，故交換



3. 檢查 60 之父點 50，比較大小，因 $60 > 50$ ，且位於 Max-Heap，故交換(完成)



分析 Time Complexity : $O(\log n)$

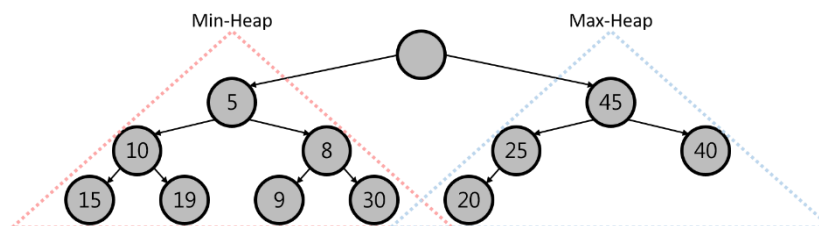
主要是花在 insert Min-Heap、Max-Heap 之調整時間，又為 Complete BT
因此高 $= \log(n+1) \Rightarrow O(\log n)$

Delete Min from Deap :

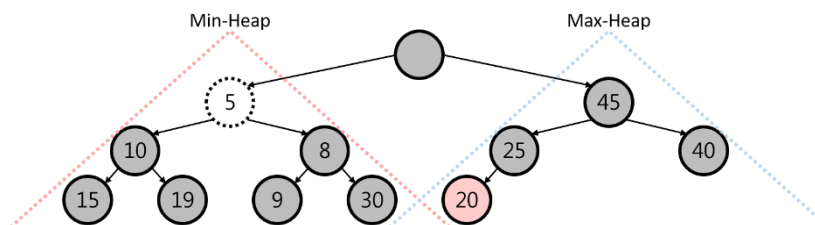
步驟：

1. 移走 Deap[2]之 Data(因為 Root 空)
2. The last Node 刪除，設定給 x
3. 左子樹 Root 空缺，由其子點最小值補上，反覆上述，直到 Leaf 形成空缺，此位置為 i
4. 執行 DeapInsert(D, i, x)

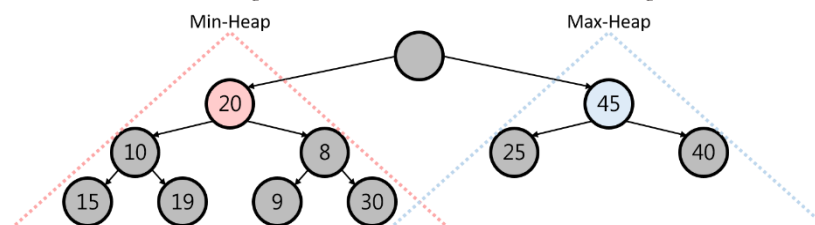
例 1：一 Deap 如圖，Delete Min 之結果為何？



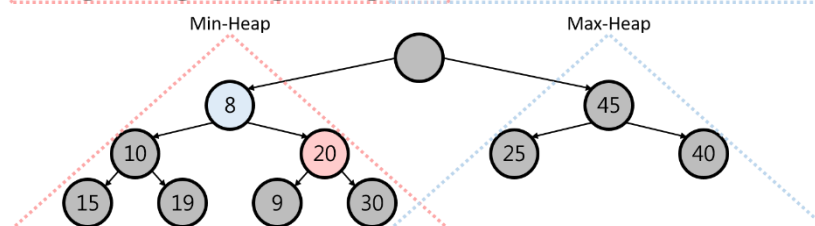
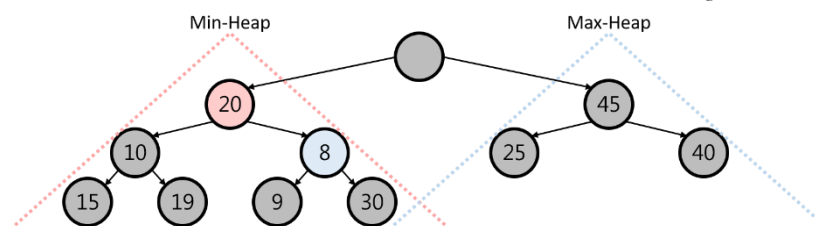
1. 刪除 Min-Heap 之 Root 5，且選出 Complete 順序最後一點 20



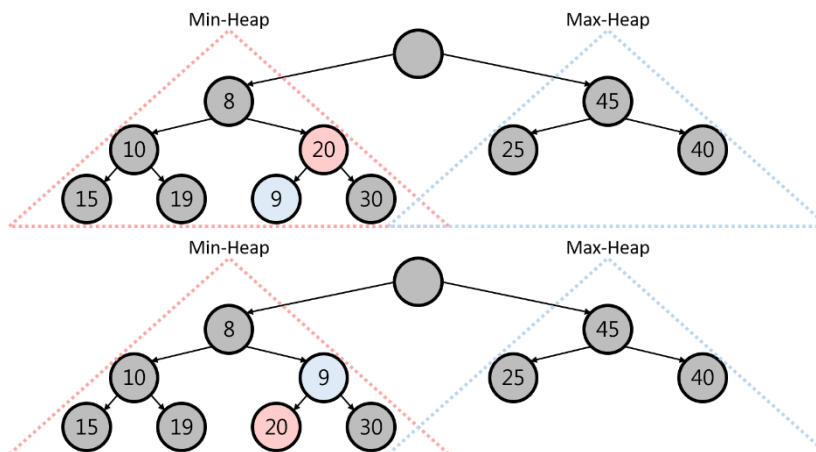
2. 將 20 放置於 Min-Heap 之 Root，且比較其對應 Max-Heap 點 45，因為 $20 < 45$ ，故不交換



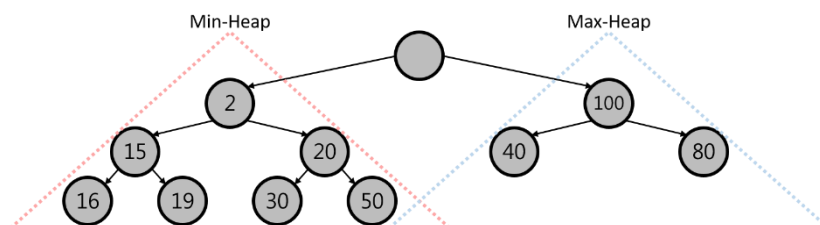
3. 選出 20 之較小子點 8，比較大小，因 $20 > 8$ ，且位於 Min-Heap，故交換



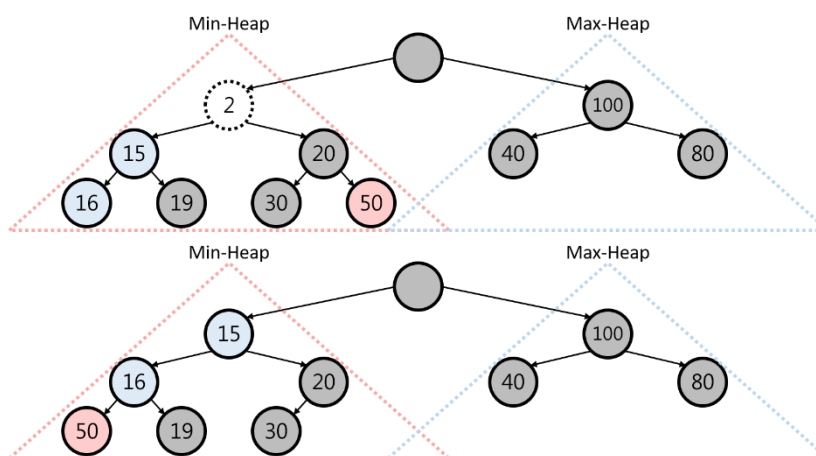
4. 選出 20 之較小子點 9，比較大小，因 $20 > 9$ ，且位於 Min-Heap，故交換(完成)



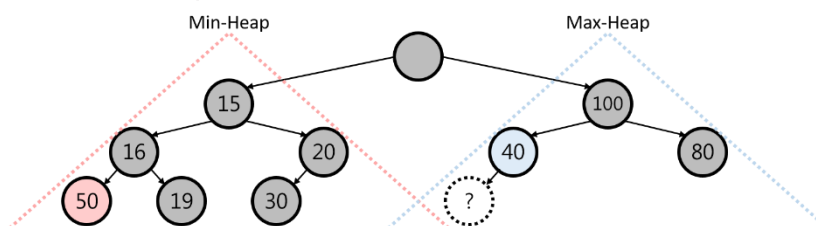
例 2：一 Deap 如圖，Delete Min 之結果為何？

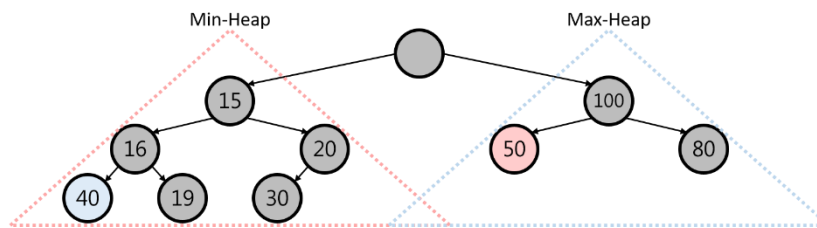


1. 刪除 Min-Heap 之 Root 2，且依序將 Min-Heap 各最小子點往上移，空出位置以 Complete 順序之最後一點 50 取代之

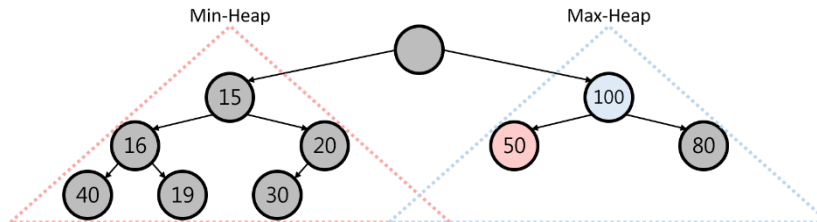


2. 比較 50 與其對應點，但因其對應點為空，故取其父點 40 比較；因為 $50 > 40$ ，故交換，將 50 移至 Max-Heap

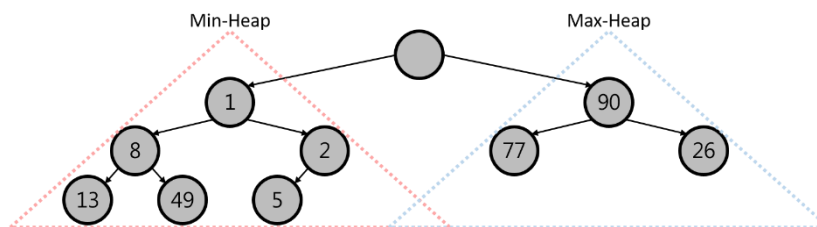




3. 檢查 50 之父點 100，比較大小，因 $50 < 100$ ，且位於 Max-Heap，故不交換(完成)



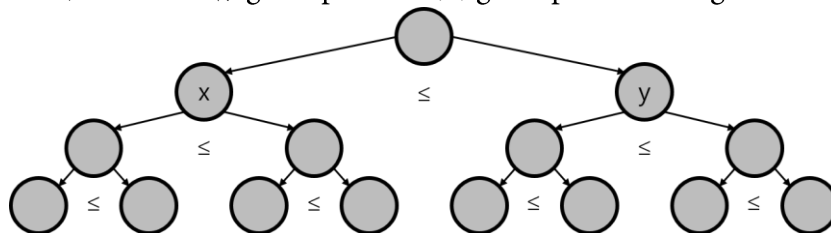
例 3：Data：26, 5, 13, 2, 8, 49, 77, 90, 1，建一 Deap？



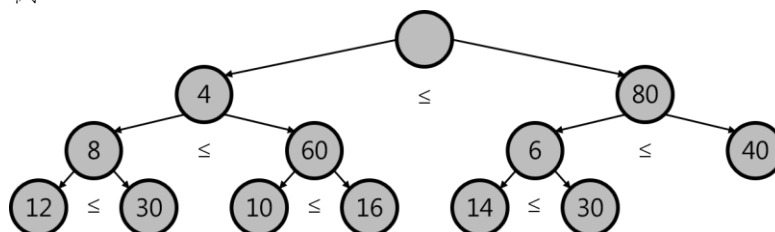
SMMH(Symmetric Min-Max Heap)

Def：是一個 Complete BT，且 Root 不存 Data，需滿足：

1. 所有 Node 之值(Root 除外)要 $\leq$ Right Sibling
2. 若 Node N 有 grandparent，則 grandparent 之 Left Child 必 $\leq N$
3. 若 Node N 有 grandparent，則 grandparent 之 Right Child 必 $\geq N$



例：



Node：另一說法：

1. Node 左子點 $\leq$ Node 子孫之值
2. Node 右子點 $\geq$ Node 子孫之值

應用：

也是製作 Double-Ended Priority Queue

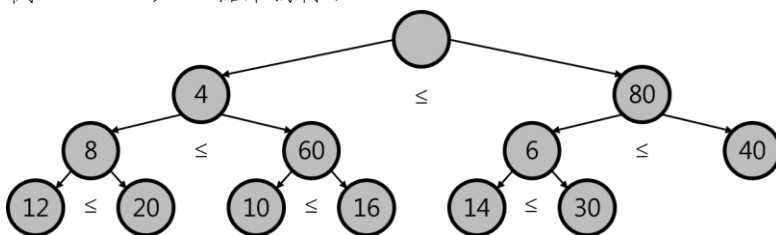
提供： $O(\log n)$ ：Insert、Delete Min、Delete Max

SMMH 之 Insert

步驟：

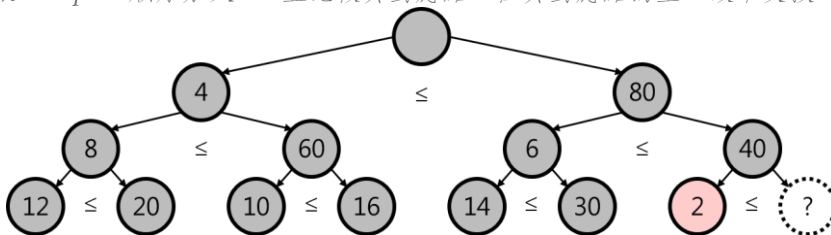
1. x 先暫置於 last Node 之下一位置，標記為 E
2. Verify p1 是否符合，不符則 E 和 Sibling 對調
3. Bottom-Up 試驗 p2、p3 是否滿足，若否則 E 上移
4. 反覆做 3.之動作直到滿足，則 x 正式放於 E

例：Insert 2, 50，結果為何？

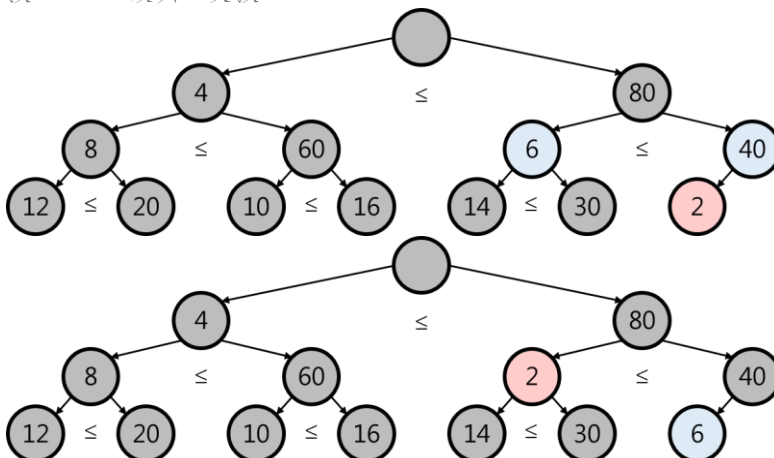


第一次 insert：

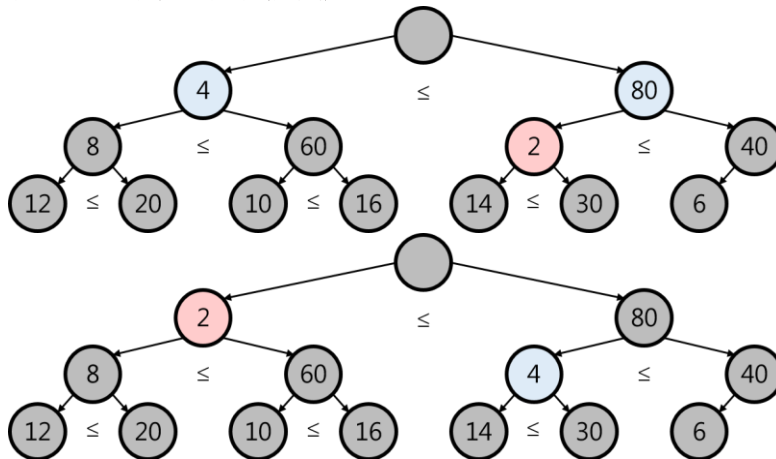
1. 依 Complete 順序加入 2，並比較其對應點，但其對應點為空，故不交換



2. 檢查 2 之父點 40 與其兄弟 6，若 2 並無介於此二者，若較小則與小交換、較大則與大交換：2 < 6，故與 6 交換

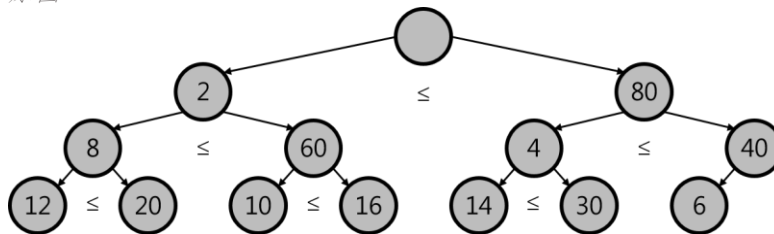


3. 檢查2之父點80與其兄弟4，若2並無介於此二者，若較小則與小交換、較大則與大交換： $2 < 4$ ，故與4交換(完成)

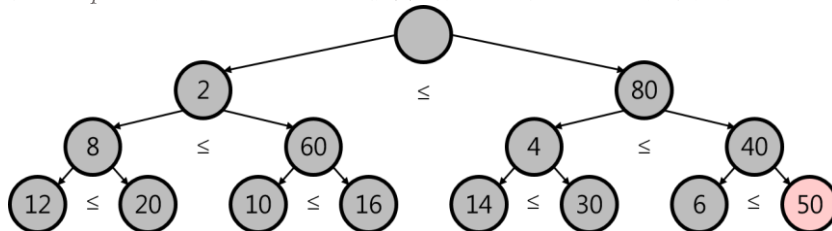


第二次 insert :

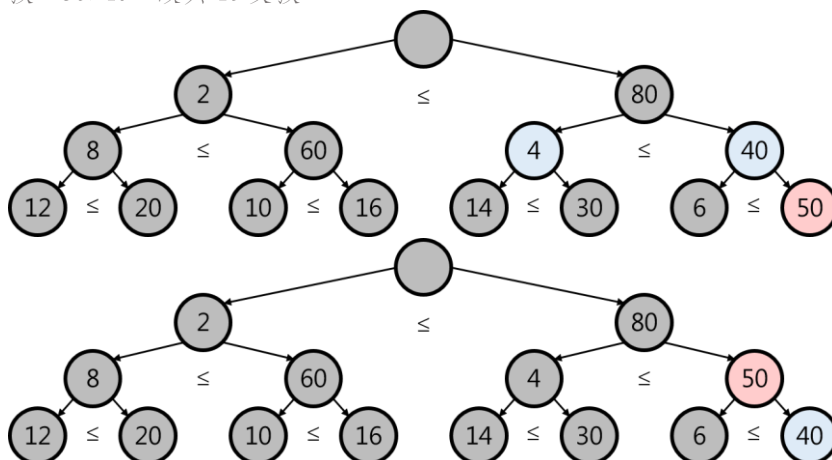
原圖 :



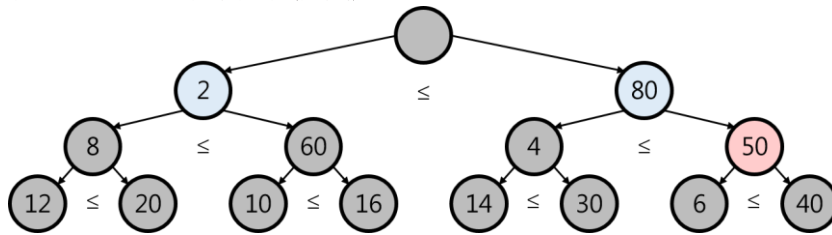
1. 依 Complete 順序加入 50，並比較其對應點，因 $50 > 6$ ，故不交換



2. 檢查50之父點40與其兄弟4，若50並無介於此二者，若較小則與小交換、較大則與大交換： $50 > 40$ ，故與40交換



3. 檢查 50 之父點 80 與其兄弟 2，若 50 並無介於此二者，若較小則與小交換、較大則與大交換： $2 < 50 < 80$ ，故不交換(完成)

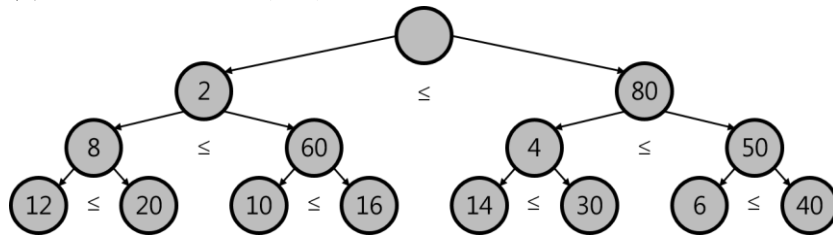


SMMH 之 Delete-Min

步驟：

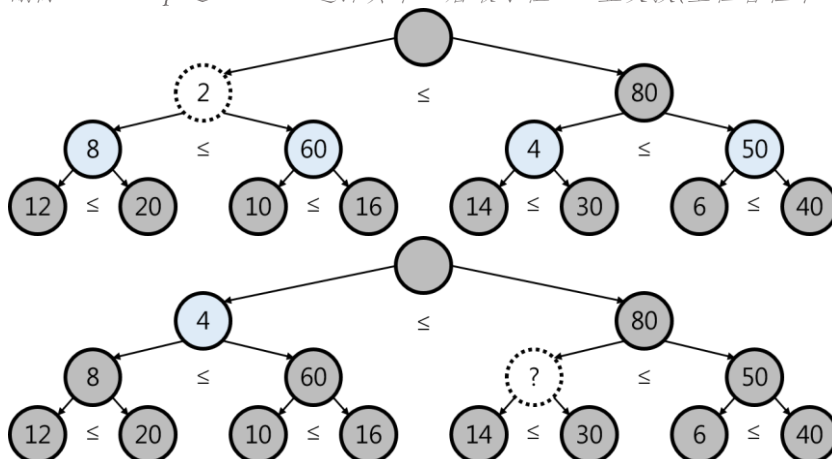
1. 移走 Root 之左子點
2. The last Node Delete，放於 x 中
3. Verify p2 是否滿足(不需 check p3，因為 Delete Min)。比較 E 的左子點和 E 的右兄弟的左子點，找最小與 E 交換
4. 反覆做 3.，直到 E 無子點，將 x 放於 E，比較 p1 是否成立，否則 swap(x, sibling)

例：Delete-Min 之結果為何？

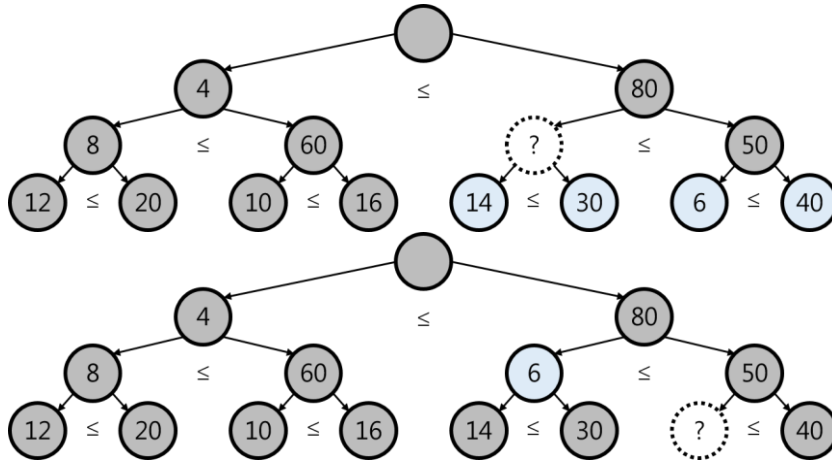


第一次刪除：

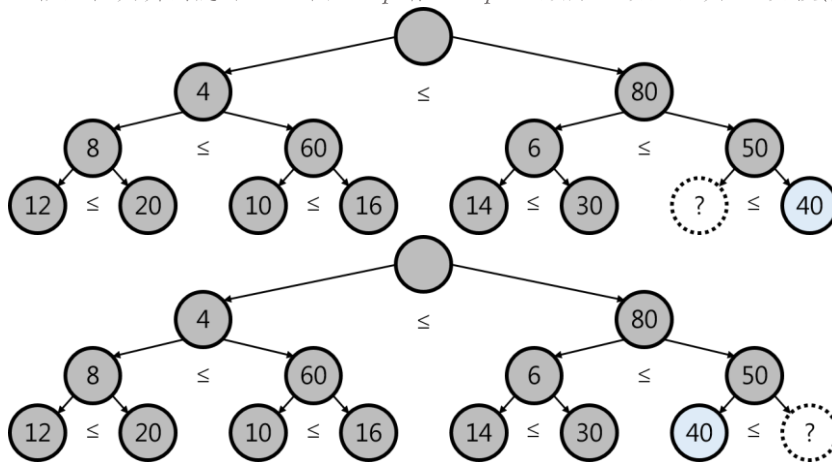
1. 刪除 Min-Heap 之 Root 2，選擇其下一層最小值 4、並交換(空位會往下一層)



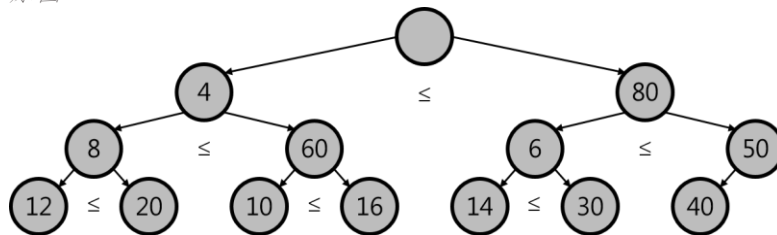
2. 選擇空位的下一層最小值6、並交換(空位會往下一層)



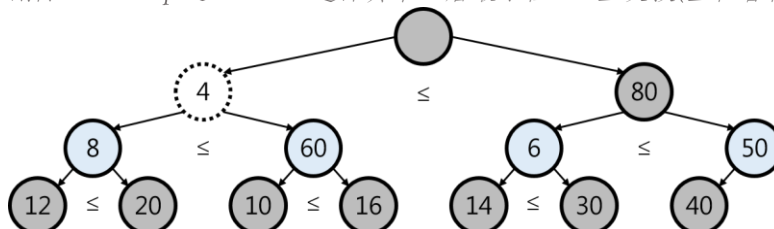
3. 比較空位與其對應點 40，因 Deap 有 Complete 順序，故空白與 40 交換(完成)

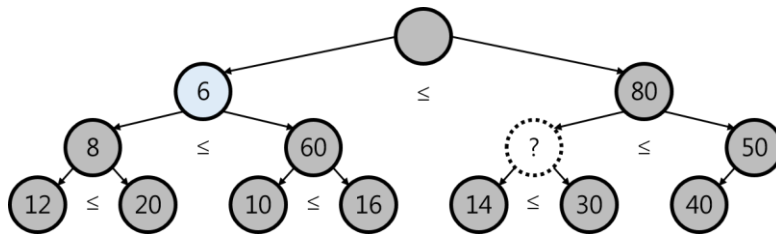


第二次刪除：
原圖：

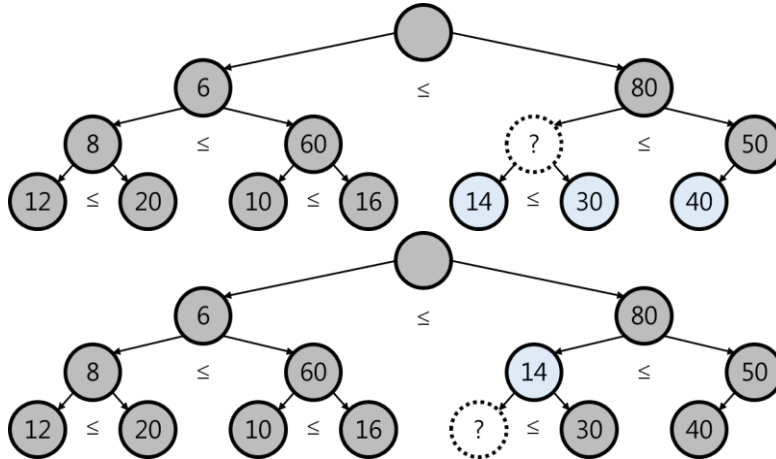


1. 刪除 Min-Heap 之 Root 4，選擇其下一層最小值 6，並交換(空位會往下一層)

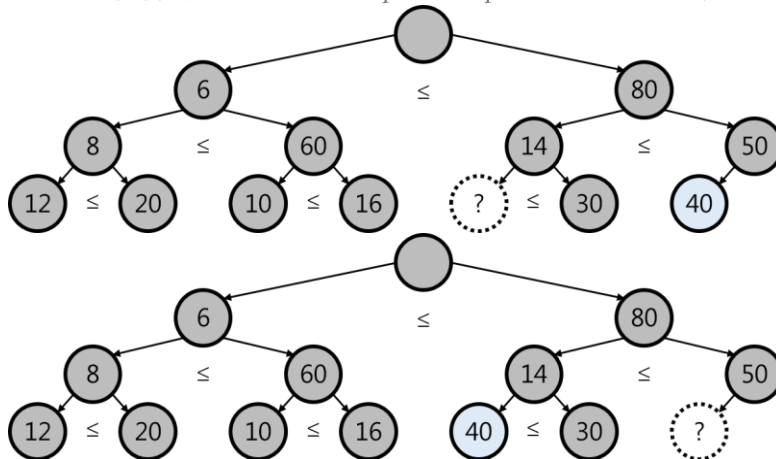




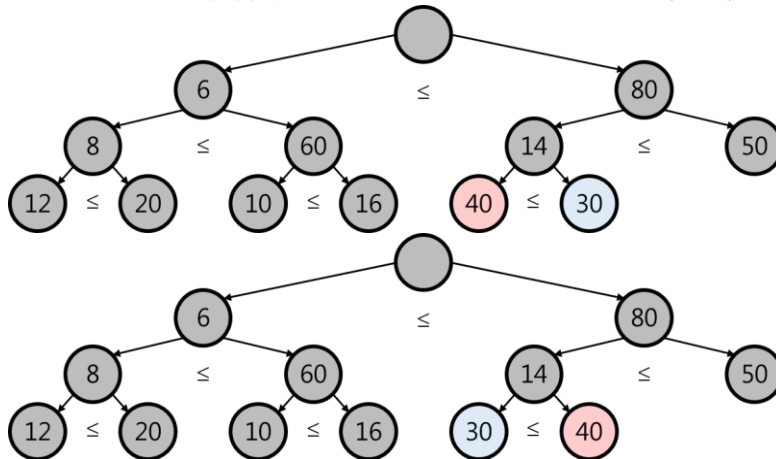
2. 選擇空位的下一層最小值、並交換(空位會往下一層)



3. 比較空位與其對應點 40，因 Deap 有 Complete 順序，故空白與 Complete 最後一點 40 交換



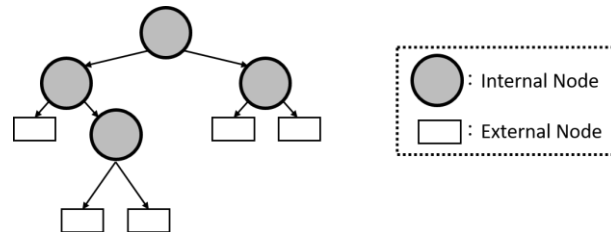
4. 交換後的 40 仍需與其對應點 30 比較，因 $40 > 30$ ，故交換(完成)



Extend BT(延伸二元樹)

Def: n 個 Node 的 BT, 以 Linked List 表示, 會有 $n+1$ 條空 Link, 在這些空 Link 上, 加上特殊 Node, 謂之”外部節點 External Node”或”失敗節點 Failure Node”

例: Extended BT



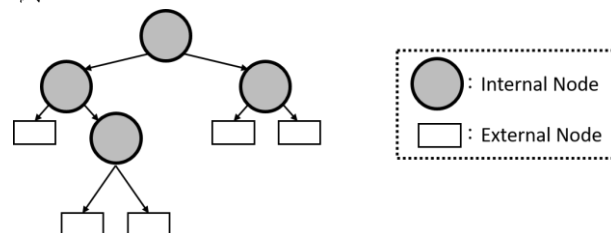
Note: 外部節點 = 內部節點+1

Internal Path Length & External Path Length(內/外部路徑長度)

$I = \sum (\text{Root 到內部節點 } i \text{ 之路徑長度})$

$E = \sum (\text{Root 到外部節點 } j \text{ 之路徑長度})$

例:



$$I = 0 + 1 + 1 + 2 = 4$$

$$E = 2 + 2 + 2 + 3 + 3 = 12$$

$$E = I + 2N$$

$$12 = 4 + 8$$

[證明] $E = I + 2N$ (n 為內部節點數), 採歸納法

1. 當內部節點數=0 個(空樹), 則 $E=I=0$, 符合 $E=I+2N$, 成立
2. 假設當內部節點數 $\leq n-1$ 時, 此定理皆成立
3. 當內部節點數= n 時, 令 L (左子樹)有:

n_L : L 的內部節點數

I_L : L 的內部路徑長度

E_L : L 的外部路徑長度

令 R (右子樹)有:

n_R : R 的內部節點數

I_R : R 的內部路徑長度

E_R : R 的外部路徑長度

在上述的假設之下, 可知整棵樹的:

$$I = I_L + I_R + n_L + n_R$$

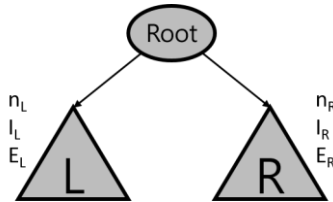
$$E = E_L + E_R + (n_L + 1) + (n_R + 1)$$

又 n_L 、 n_R 均 $\leq N-1$ ，依 2 的假設可知：

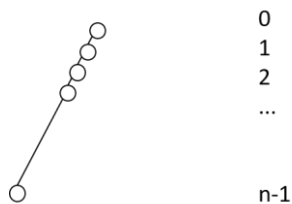
$$E_L = I_L + 2n_L$$

$$E_R = I_R + 2n_R$$

$$\begin{aligned} \text{故 } E &= E_L + E_R + (n_L + 1) + (n_R + 1) \\ &= I_L + 2n_L + I_R + 2n_R + n_L + 1 + n_R + 1 \\ &= I + (2n_L + 2n_R + 1) \\ &= I + 2n \dots \text{故成立，且由 1、2、3 得證} \end{aligned}$$



例：在一 Skewed Extended BT 是 n 個內部節點，求 I 為何、 E 為何？



$$I = 0 + 1 + 2 + \dots + n - 1$$

$$E = I + 2N = n(n-1)/2 + 4n/2 = n(n+3)/2$$

結論：

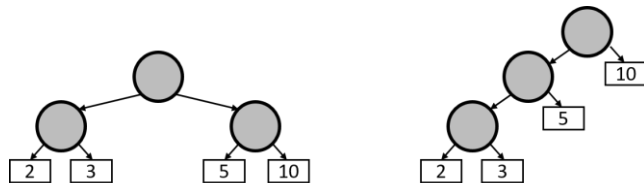
1. E 、 I 成正比
2. 在沒有內、外部加權值下，高度越低， E 跟 I 越小

Weight External Path Length(加權外部路徑長度 WEPL)

Def：給予 $(n+1)$ 個外部節點加權值，令為 $g_i (0 \leq i \leq n)$ ，則

$$WEPL = \sum [(Root \text{ 到外部 Node } j \text{ 之長度}) * g_i]$$

例：



$$WEPL = 2*2 + 2*3 + 2*5 + 2*10 = 40$$

$$WEPL = 3*2 + 3*3 + 2*5 + 1*10 = 35$$

由上可知，在有外部加權值下，平衡不見得可使 I 和 E 越小(平衡不見得好)

Minium WEPL 如何求得？

Minimum Weight External Path Length(最小加權外部路行長度 MWEPL)

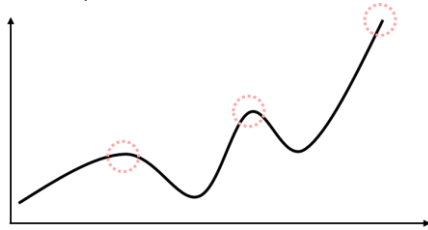
Def: 給予 $n+1$ 個外部節點加權之下，在 $\frac{1}{n+1}C_n^{2n}$ 棵不同 BT 中，找最小的 WEPL 值謂之

[法一]: 用 Brute Force(暴力法)

將所有 $\frac{1}{n+1}C_n^{2n}$ 棵不同 BT 之 WEPL 求出，取最小值 $\Rightarrow O(2^n)$ (顯然不可能採用)

[法二]: 採用”Huffman 演算法”

策略: 採 Greedy Algorithm(貪婪演算法)，即每回合都挑當下的最佳解，But Greedy 的結果針對不同問題，不見得皆為 Global Optimal Solution



(只能求出局部最佳解)

若要求全域最佳解，需採 Dynamic Programming

[補充]Note: 採 Greedy 可得最佳解的:

1. Huffman
2. Minimal Cost Spanning Tree(Kruskal's、Prim's、Sollin's)
3. One to all Shortest Path(Dijkstra's)
4. Fractional Knapsack Problem(背包問題的一種)

註: 另一種背包問題(0/1): 不能用採 Greedy，需用 Dynamic Programming

Huffman 霍夫曼演算法

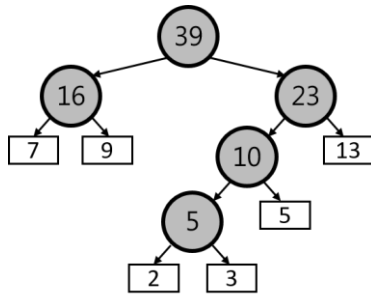
題型:

1. 求 MWEPL
2. 建”Encoding Tree”
3. 平均最短編碼長度

步驟:

1. 將外部加權值放到集合 S 之中
2. 每回合從 S 取 2 個最小值 x, y ，並將之刪除，以 $x+y(\text{sum})$ 為 x, y 的 parent node
3. 若 S 非空將 sum 放回 S 之中
4. Repeat 2.和 3.直到 S 為空

例 1 : 2, 3, 5, 7, 9, 13 , 求 MWEPL ?



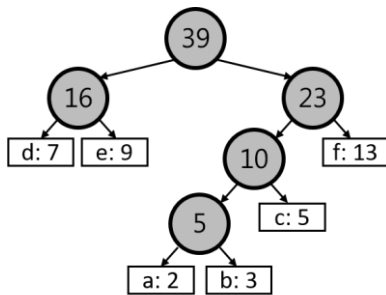
$$MWEPL = 2*4 + 3*4 + 5*3 + 7*2 + 9*2 + 13*2 = 92$$

Note : Huffman Tree 不見得唯一

例 2 : 字母出現的頻率為 : (a, b, c, d, e, f) = (2, 3, 5, 7, 9, 13)/39 , 問 :

1. 平均編碼長度最短之 Encoding Tree ?
2. 各字的編碼 ?
3. 平均最短時傳一字母的 bits 數 ?

1.



2. 表

a	b	c	d	e	f
1000	1001	101	00	01	11

3. $93/39 = 2.38 \text{ bits}$

例 2 : 承上，今送出一 Message 1000000111 為何 ?

1000 00 01 11 => adef

程式 :

```

Huffman(l, n)          //l 為外部加權集合，n 為 l 的大小
{
    for i=1 to n-1 do
    {
        new(t);
        t->Lchild = least(l);
        t->Rchild = least(l);
        t->weight = (t->Lchild)->weight + (t->Rchild)->weight;
        Insert(l, t);    //將 t 放回 l
    }
}
  
```

分析：

共需做 $n-1$ 回合

各回合做 2 個 operation：

Least(l) => 從 l 取最小

Insert(l, t) => t 加回 l

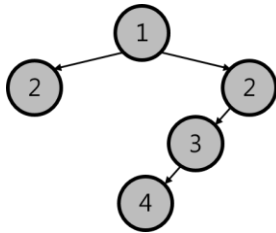
又 l 集合適用 Heap 做，故

Least(l) Insert(l, t) $\Rightarrow O(\log n)$

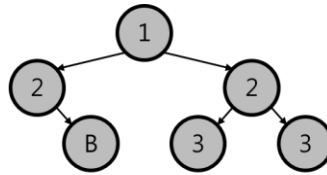
Delete insert 跟樹高相關

故 Time Complexity = $(n-1) * (3 * O(\log n)) \Rightarrow O(n \log n)$

如何評估 BST 優劣？



圖一： $(1+2+2+3+4)/5=2.4$



圖二： $(1+2+2+3+3)/5=2.2$ (較佳)

結論：在沒有加權值的情況下，愈平衡愈好

$\Rightarrow$ 採用 "Balance BST"，謂之 AVL Tree

AVL Tree

概念：動態維持平衡，故高度為 $O(\log n)$

因此：Insert、Delete、Search： $O(\log n)$

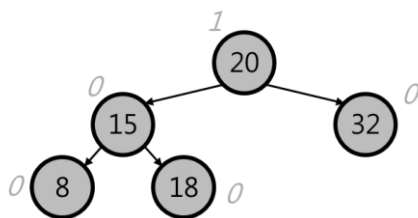
在先前 BST 在處理 Dynamic Data，若變成 Skewed BT $\Rightarrow O(n)$

$\Rightarrow$ AVL Tree 會解決此問題

Def：為一棵 "平衡的 BST"，若 AVL Tree 不為空，會滿足：

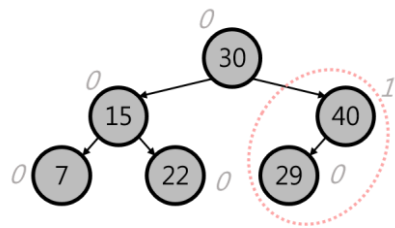
1. $|hL - hR| \leq 1$
2. 左、右子樹亦為 AVL Tree
3. AVL Tree 中的平衡系數 (Balance factor：-1、0、1，即是 $|hL - hR|$)

例 1：



是 AVL Tree

例 2 :



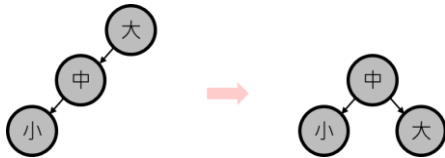
圓圈內根本沒滿足 BST 之原則，故不是 AVL Tree

AVL Tree 不平衡的調整方式

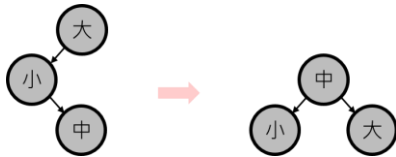
1. LL
2. LR
3. RL
4. RR

概念：中間值往上拉、大的擺右、小的擺右

1. LL



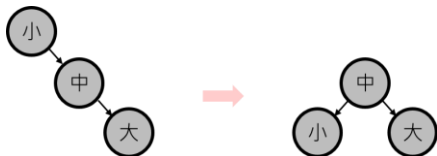
2. LR



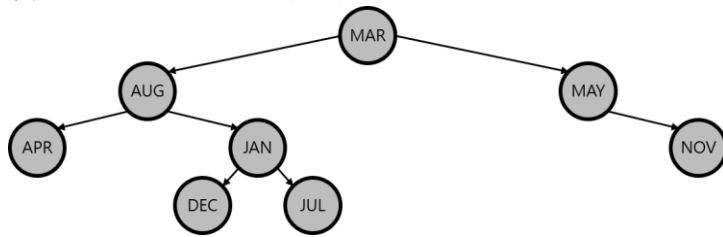
3. RL



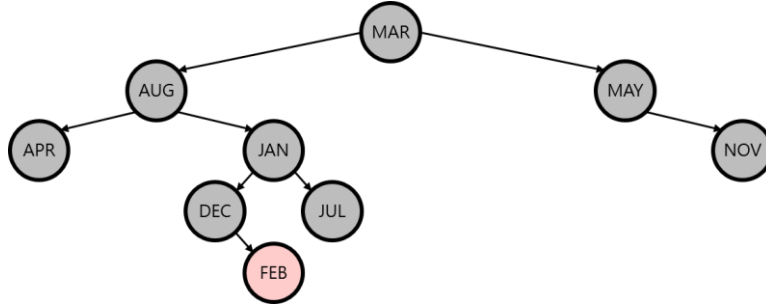
4. RR



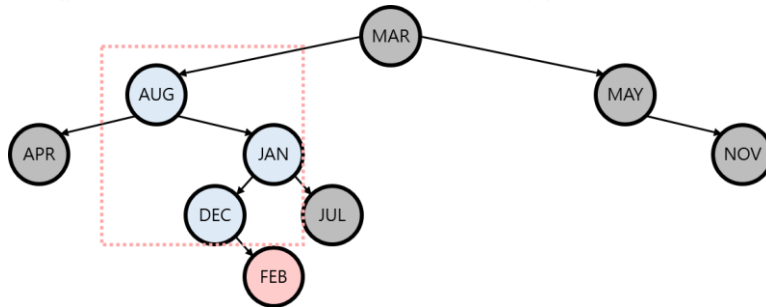
例 1：Insert “Feb”之結果為何？



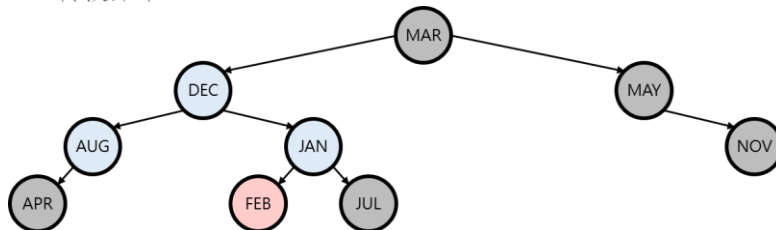
1. 插入 FEB 於正確位置(依 BST)



2. 檢查 AVL 順序，只需轉換方框內，便可平衡



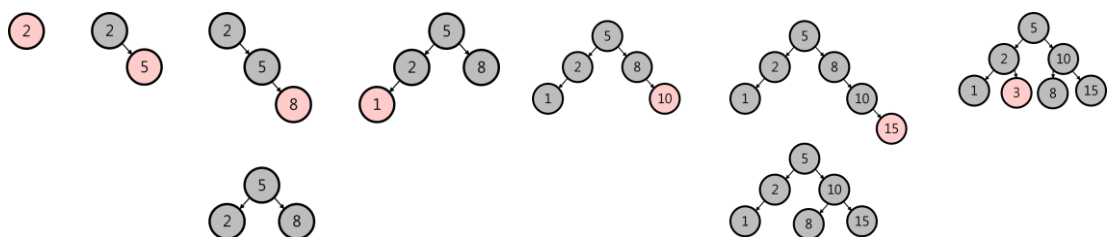
3. 轉換結果



動態維持平衡

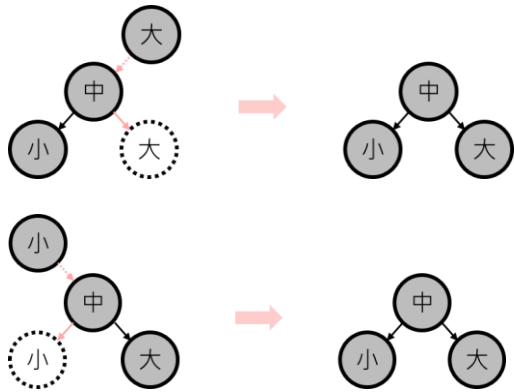
例 2：給予 2, 5, 8, 1, 10, 15, 3，建立一 AVL Tree？

步驟 1~6：

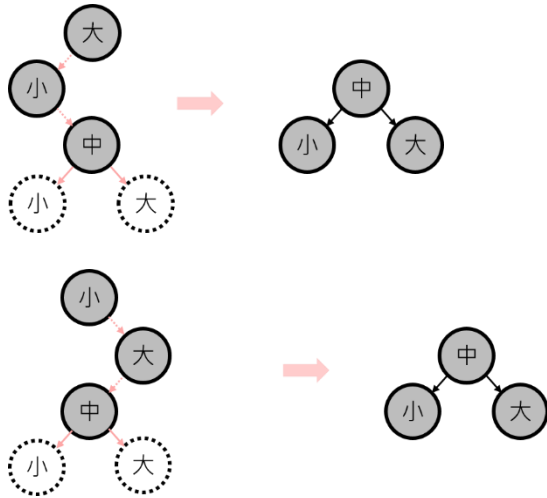


例 3 :

1. LL、RR 調整 => 改 2 個指標

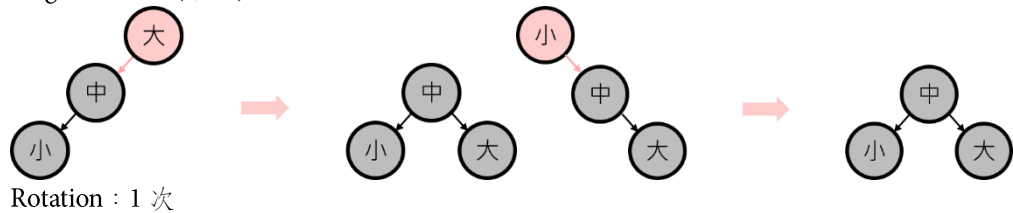


2. LR、RL 調整 => 改 4 個指標



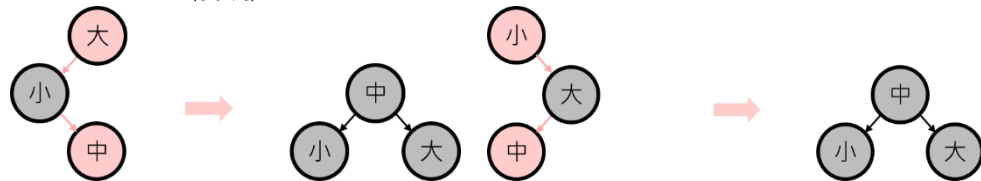
3. 就演算法角度而言

(1) Single Rotation(單旋) : LL、RR



Rotation : 1 次

(2) Double Rotation(雙旋) : LR、RL



Rotation : 2 次

AVL 定理：

欲形成高度為 h 的 AVL Tree，所需之最少 Node 數： $F_{h+2} - 1$ (F 為費氏數列)

n	0	1	2	3	4	5	6	7
Fn	0	1	1	2	3	5	8	13

Note：最多 Node 數： $2^h - 1$

例 1：形成高度為 5 之 AVL Tree

1. 最少 Node 數為何？
2. 最多 Node 數為何？

1. 12 ($F_7 - 1$)
2. 31 ($2^5 - 1$)

例 2：[反向]AVL Tree 有 15 個 Node

1. 最小高度為何？
 2. 最大高度為何？
1. 4
 2. 5 ($F_7 - 1$ 最後一個滿足的值)

例 3：AVL Tree 有 300 個節點，最大高度為何？

13

[證明]：數學歸納法可證(略)

小結：Array、Linked List、AVL Tree

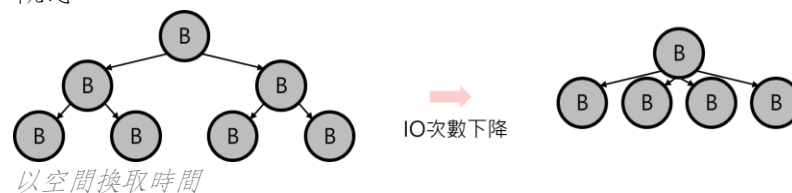
Operation\DS	Array	Linked List	AVL Tree
1. Search for x	$O(\log n)$ 用 Binary Search	$O(n)$	$O(\log n)$
2. Search for kth item	$O(1)$	$O(k)$	$O(\log n)$
3. Delete kth item	$O(k-k)$	$O(k)$	$O(\log n)$
4. Insert x	$O(n)$	$O(1)$	$O(\log n)$
5. Output in order	$O(n)$	$O(n)$	$O(n)$

m-way Search Tree(Tree Degree： $m > 2$)

緣由：在面對 External Search，欲降低 Search Time，則必須降低 Disk IO 次數，即”降低 Search Tree 之高度”

⇒ 有效的方式則是”放大 Degree”

概念：



B-Tree

Def: 為 Balance m-way Search Tree, 若不為空, 滿足:

1. Root 至少有 2 個 Child
2. 除 Root 跟 Leaf, 其餘的 Degree 介於 $\lceil m/2 \rceil \sim m$ 之間
3. 所有的 Leaf(Failure) Node 會在同一層

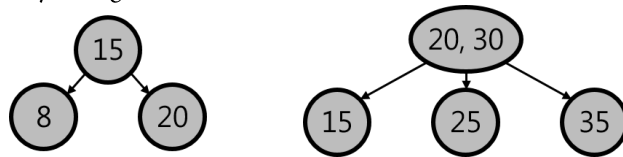
例: 2-3 Tree

3 way Search Tree

B tree of order 3(m)

Degree 需介於 $\lceil 3/2 \rceil \sim 3 \Rightarrow 2 \sim 3$

Key = Degree-1



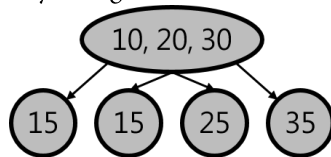
例: 2-3-4 Tree

4 way Search Tree

B tree of order 4(m)

Degree 需介於 $\lceil 4/2 \rceil \sim 4 \Rightarrow 2 \sim 4$

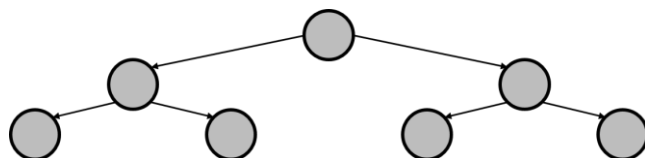
Key = Degree-1



例: 高度為 h, Degree=m 之 B-Tree, 問:

1. 最多 Node 數為何?
2. 最多 key 數為何?
3. 最少的 Node 數為何?
4. 最少的 key 數為何?

1. $m^0 + m^1 + m^2 + \dots + m^{h-1} = (m^{h-1+1} - m^0)/(m-1) = (m^h - 1)/(m-1)$
2. $(m^{h-1})/(m-1) * (m-1) = (m^h - 1)$
3. $1 + 2*\lceil m/2 \rceil^0 + 2*\lceil m/2 \rceil^1 + 2*\lceil m/2 \rceil^2 + \dots + 2*\lceil m/2 \rceil^{h-2}$
 $= 1 + 2*([\lceil m/2 \rceil^0 + \lceil m/2 \rceil^1 + \lceil m/2 \rceil^2 + \dots + \lceil m/2 \rceil^{h-2}) = 1 + 2*([\lceil m/2 \rceil^{h-1} - 1)/(\lceil m/2 \rceil - 1)$
4. $1 + 2*([\lceil m/2 \rceil^{h-1} - 1)/(\lceil m/2 \rceil - 1) * (\lceil m/2 \rceil + 1) = 1 + 2*([\lceil m/2 \rceil^{h-1} - 1) = 2*\lceil m/2 \rceil^{h-1} - 1$



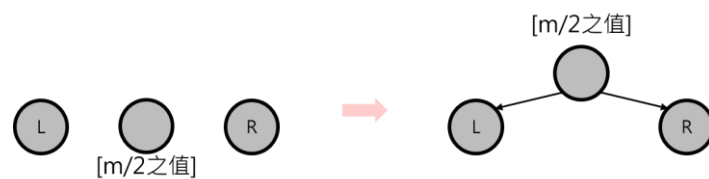
B-Tree 之 Insert, Delete(主要以 2-3 Tree 為例)

Insert :

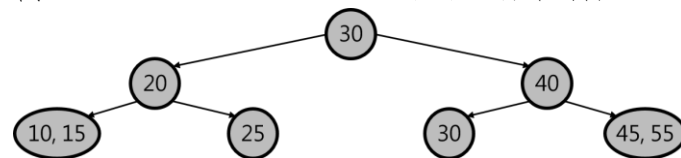
步驟：

1. 找到正確的插入位置
2. 將欲插入值 x ，插入
 - (1) 無 overflow(key 數 $\leq m-1$)，停止
 - (2) 有 overflow(key $>m-1$)，做”Split”處理

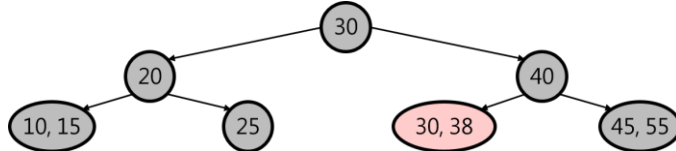
Split(Bottom-Up 方式)：將第 $\lceil m/2 \rceil$ 個 key 往上拉到父點，其餘分成 2 個 Node
概念：



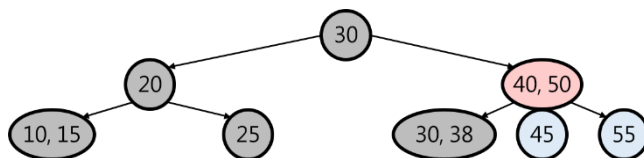
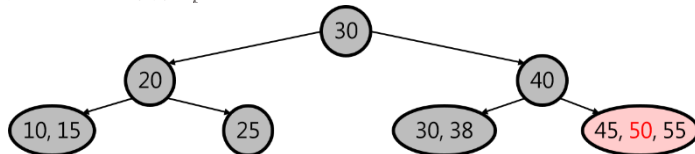
例 1：B-Tree of order 3，insert “38, 50, 37”結果為何？



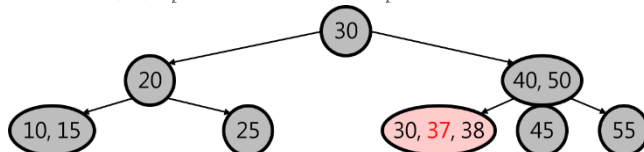
1. 加入 38

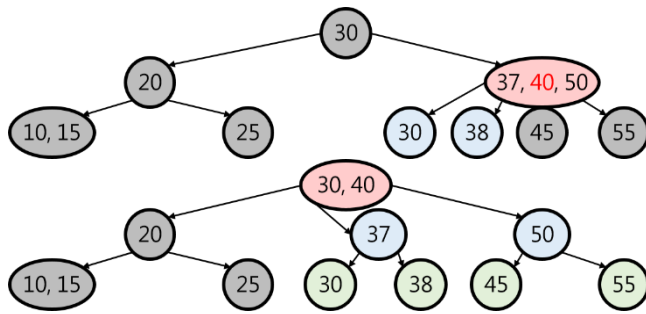


2. 加入 50，進行 Split

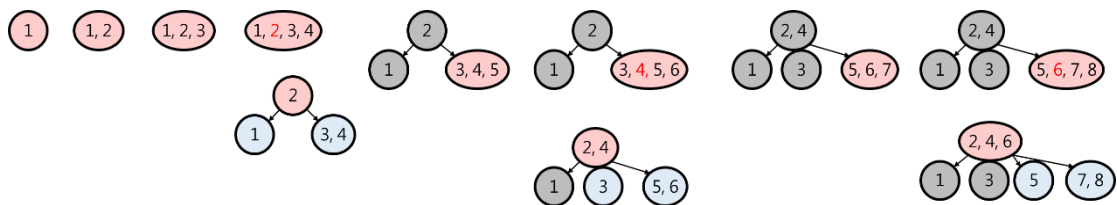


3. 加入 37 進行 Split，再往上進行 Split





例 1 : Data : 1, 2, 3, 4, 5, 6, 7, 8 , 建立一 2-3-4 Tree



B-Tree 的 Delete

步驟：

1. 找到欲 Delete Node(x)所在
2. Check x 所在：
 - (1) 於 Leaf => Delete
 - 若 key 數 $\geq \lceil m/2 \rceil - 1 \Rightarrow$ No underflow
 - 若 key 數 $< \lceil m/2 \rceil - 1 \Rightarrow$ 有 underflow
 - (2) x 於 Non-leaf Node ,
 - 則找左子樹最大值取代 x ；
 - 或找右子樹最小值取代 x
 - $\Rightarrow$ 被挑中 Leaf 設為 x , goto 2

Underflow 處理方式：

1. 先做 Rotation
2. 若無法 Rotation , 再做 Combination

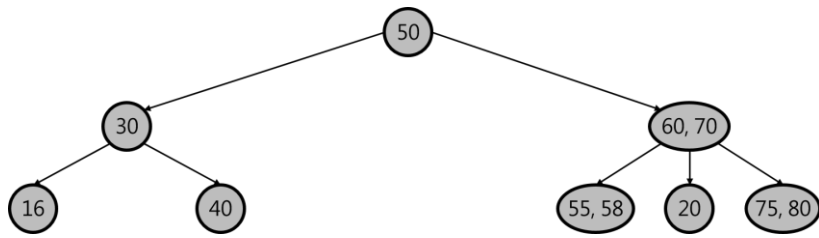
Note :

Rotation : 跟相鄰兄弟 Node 藉一個 key

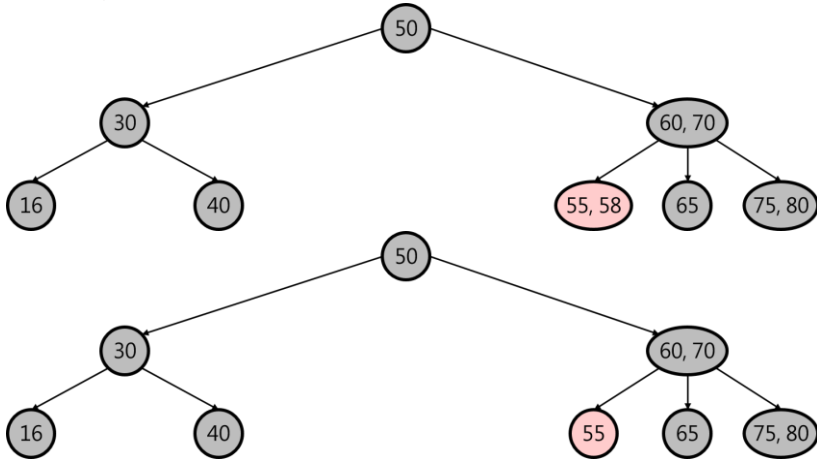
Combination : 將 Parent 跟 Sibling 合併成一個 Node

例 1 : B-Tree order 3

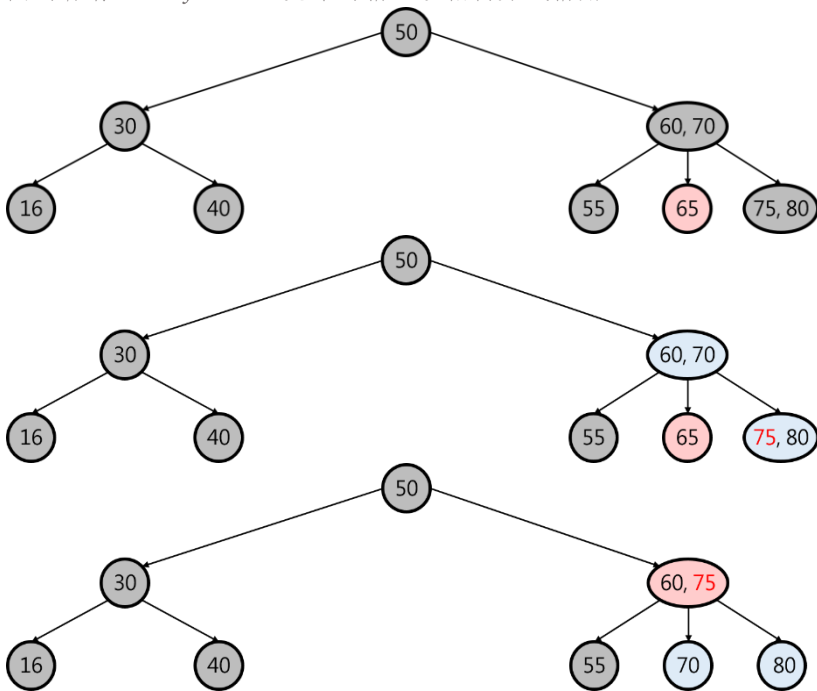
1. Delete 58
2. Delete 65
3. Delete 55
4. Delete 50
5. Delete 80



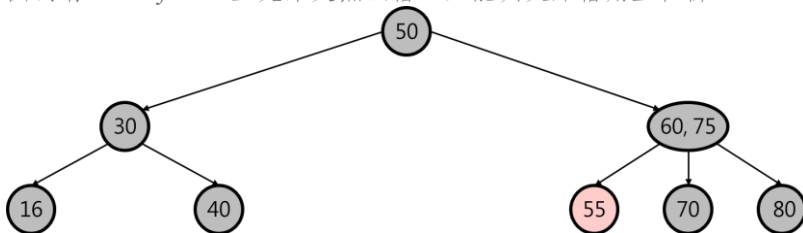
1. 無 Underflow，直接刪除

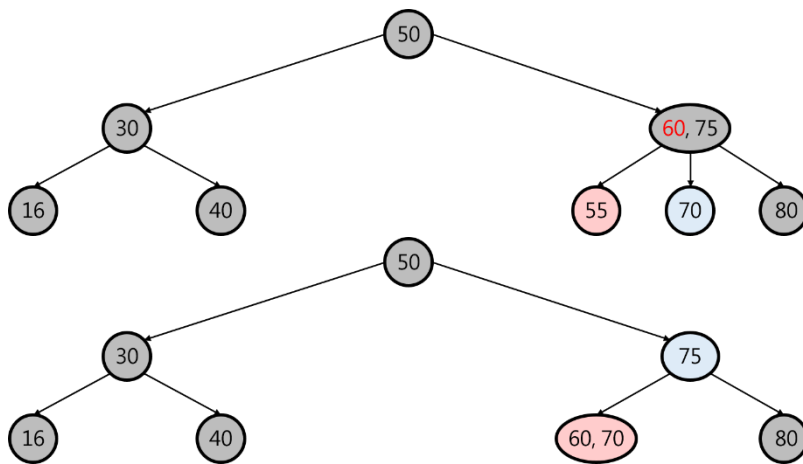


2. 因為會有 Underflow，且兄弟可借，以輪轉方式借用

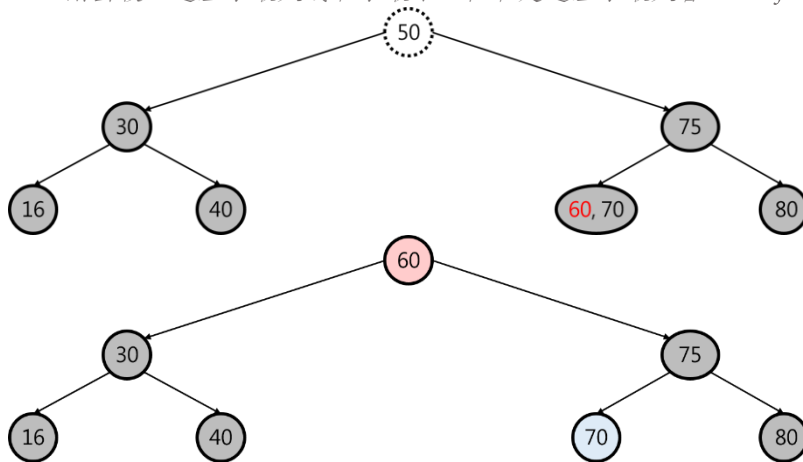


3. 因為有 Underflow，且兄弟又無法借，只能與父點借用並合併

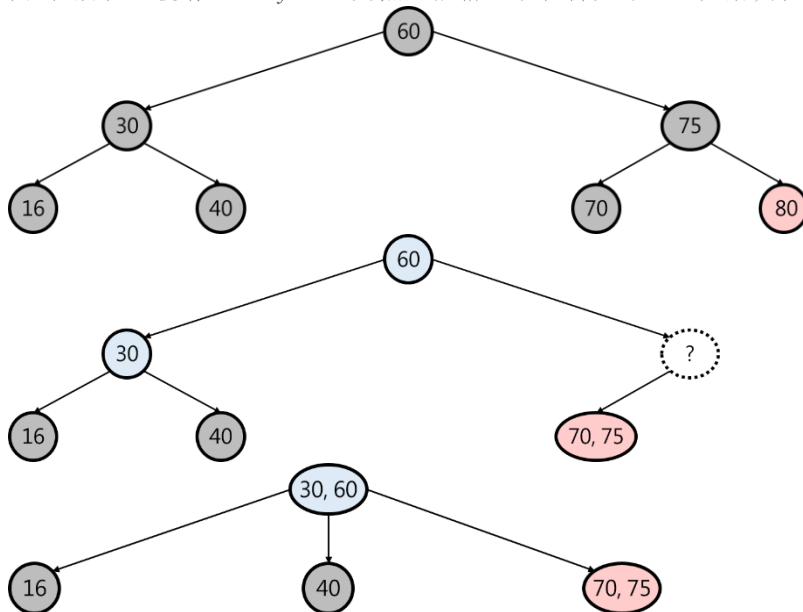




4. Root 刪去後，選左子最大或右子最小，但若是選左子最大會 Underflow，故選右子最小

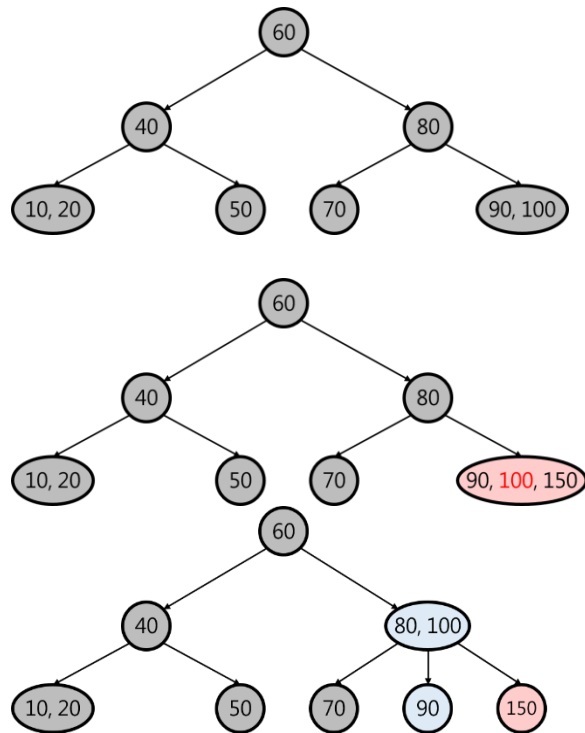


5. 因為刪去 80 後有 Underflow，又無人能借，故合併父點，且父層與祖父層重新合併



[變化題]

例 1：若 Insert 150 後，問 Disk 的 IO 次數為何？

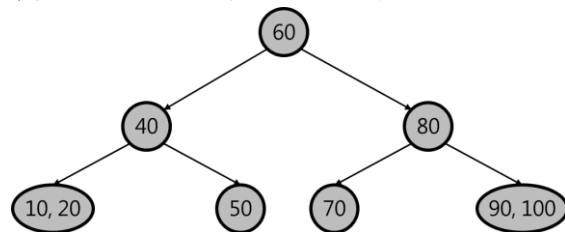


Read : a, b, c => 3 次

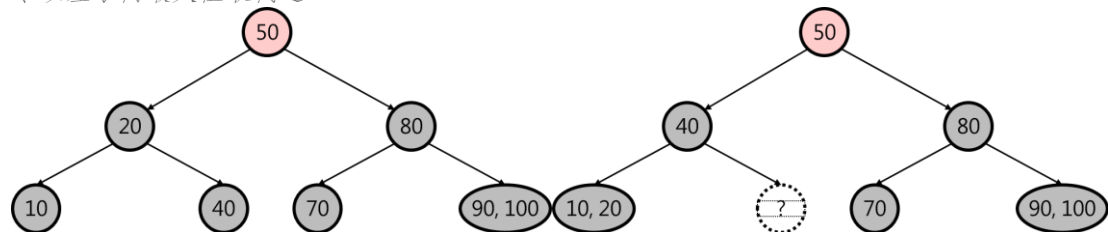
Write : 3 次

⇒ 3+3 = 6 次

例 2：Delete 60，問 Disk IO 次數為何？



令以左子樹最大值取代之



Read : a, b, c, d => 4 次

Write : 4 次

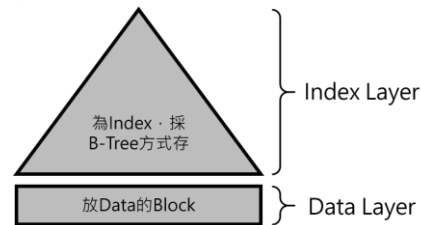
⇒ 4+4 = 8 次

B⁺ Tree of order m(於 Database 中常用)

Def :

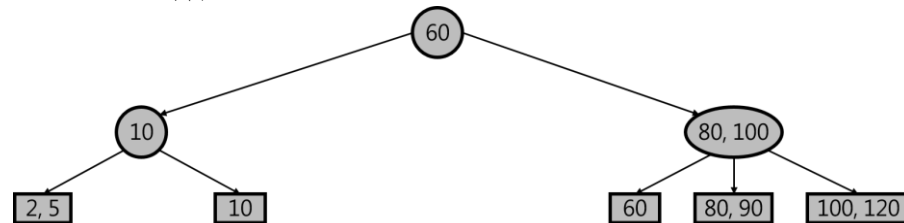
1. 主要用於 External Search/Sort
2. Support ISAM(Index Sequential Access Method)實施
3. 由 Index Layer 和 Data Layer 組成

概念 :

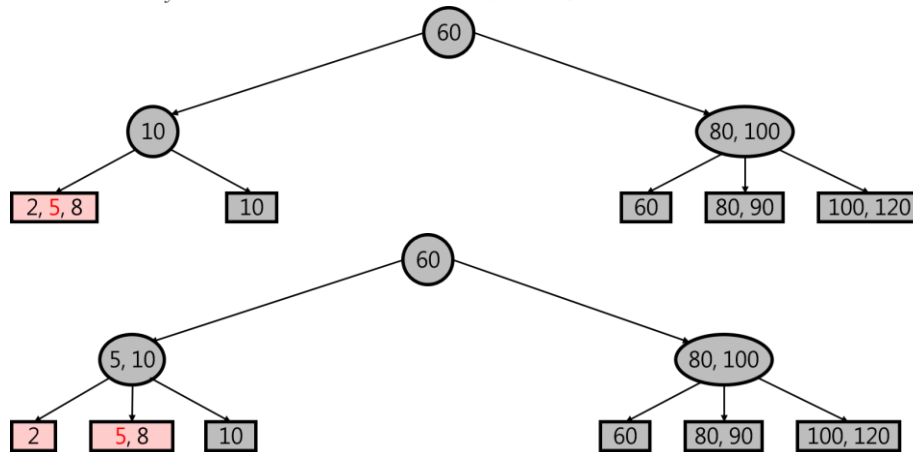


例 1 : B⁺ Tree of order 3

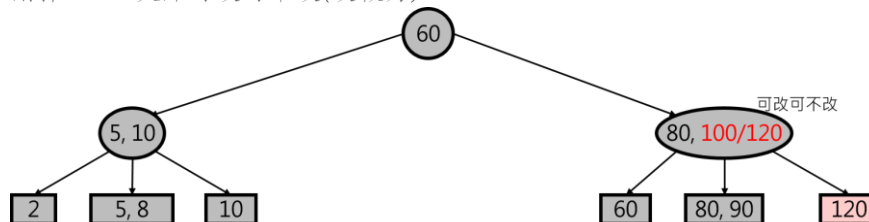
1. insert 8 結果為何 ?
2. Delete 100, 10, 5 為何 ?
3. Delete 8 為何 ?



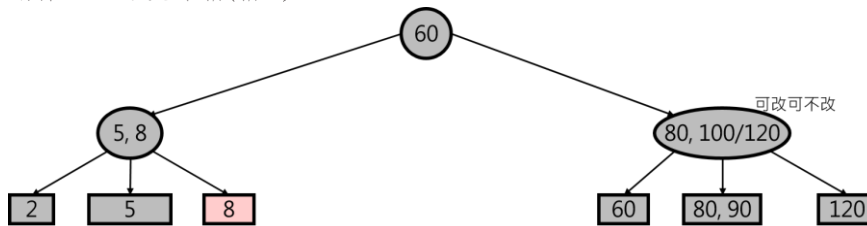
1. Insert 8 · Overflow , 故 Convolution , 但 5 不能直接拉上去 , 亦需留一在 Data 層



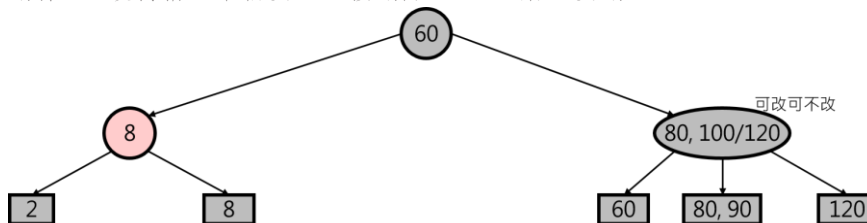
2. 刪掉 100 , 父點可改可不改(改較好)



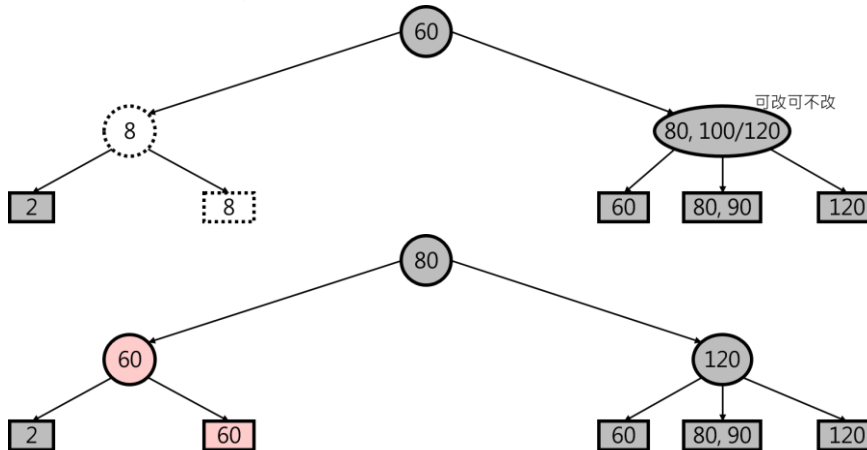
刪掉 10，跟兄弟借(借 8)



刪掉 5，沒得借也不需要，直接刪除，Index 層也要刪



3. Index 調整和 B-Tree 相同，重建後再把 Data 層連上、補上 Data



比較表

B-Tree	B ⁺ Tree
Non-Leaf，有 Data	Data 皆在 Leaf Non-leaf 為 Index
Insert, Delete 較麻煩 (因為 Data 可能在 Non-leaf)	Insert, Delete 較容易
實作較麻煩	實作較簡單
不需 Index => 沒有 ISAM 的機制	需 Index(耗 Space) => Support ISAM(速度較快)

Red & Black Tree 紅黑樹

緣由：使得其 Rotation 次數較 AVL Tree 少

⇒ 即讓調整的效能提升

[版本]有 2：

1. DS
2. 演算法版(較常見)

[DS 版]

Def：是 2-3-4 Tree 對應的 BST

此 Link 出現在原 2-3-4 Tree 時為黑，否則為紅

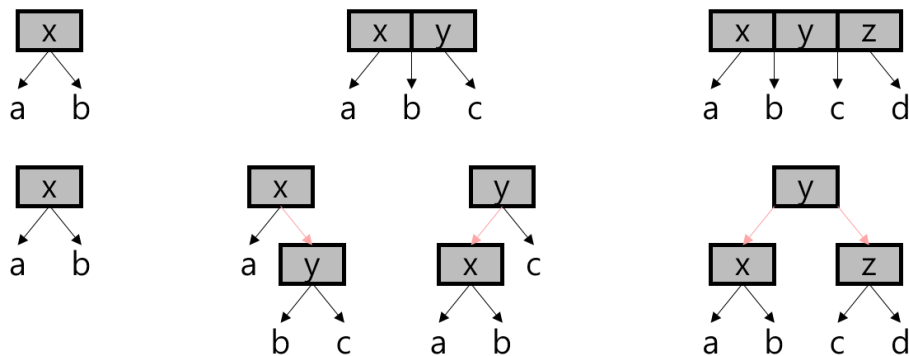
任何路徑上不得存在連續 2 個紅 Link

由 Root 到任何 Leaf 路徑上，黑 Link 相同 => 反應 B-Tree 中，所有失敗的 Node 皆在同一層

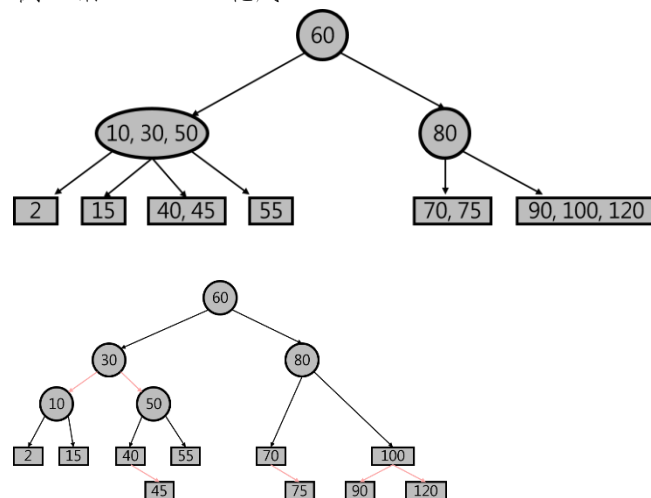
2-3-4 Tree 化成 RB Tree，原高 $O(\log n)$ ，轉換之後：

最低： $O(\log n)$

最高： $2 \cdot O(\log n)$



例：將 2-3-4 Tree 化成 RB Tree



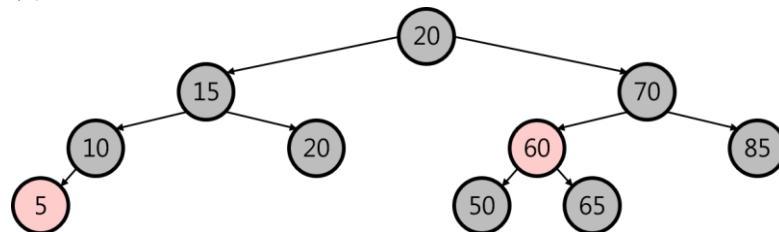
Note：結果可能不唯一(考試時要統一：小左 or 大右，選其中一種)

[演算法版]

Def: 為一 BST, 且滿足:

1. Node 的 Color 非紅即黑
2. Root 一律為"黑"
3. Null 亦視為"黑"
4. 若為紅節點, 其子點為黑(即不可連續的紅 Node 存在)
5. Root 到任何 Leaf 其經過的黑 Node 數皆相同

例:



操作: Insert/Construction

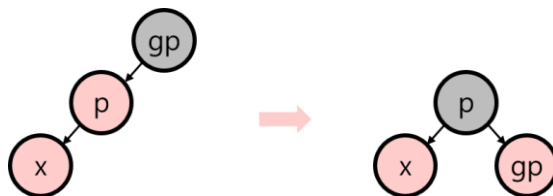
1. Top-Down(Simple)
2. Bottom-Up(Complexity, 不談)

Top-Down 做法:

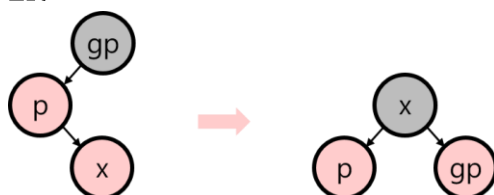
1. 尋找 x 適當的插入位置
2. 在 Search 路徑中, 遇 2 個紅色子點時, 需要:
 - (1) Color Change(子點變黑)
 - (2) 之後 Check 有無連續的紅色 Node, 若有, 需 Rotation
3. 置入 x, 且標為紅色 Node
4. Check 有無連續 2 紅、有的話, 需要 Rotation
5. Root 一律改為黑色(if needed)

Note: Rotation 同 AVL Tree 調整(但父點黑, 子點紅)

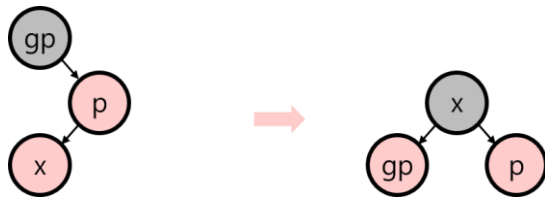
1. LL



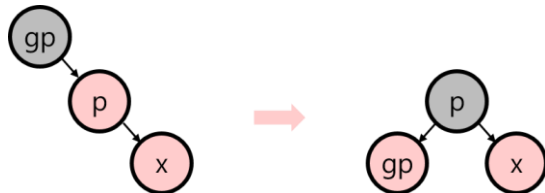
2. LR



3. RL

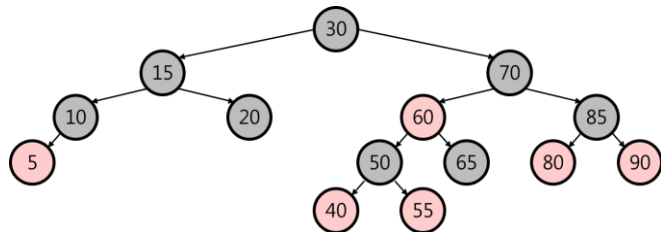


4. RR

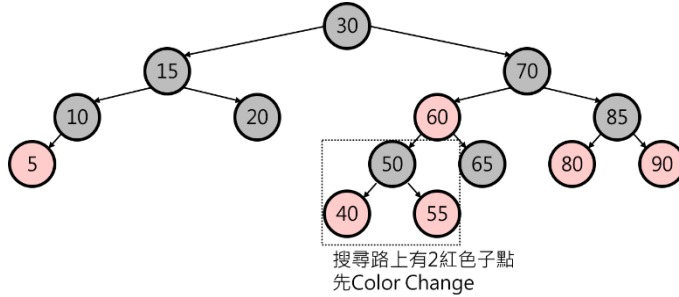


Note：考試時要定義清楚，可用 1 圖與 2 圖表示紅、黑

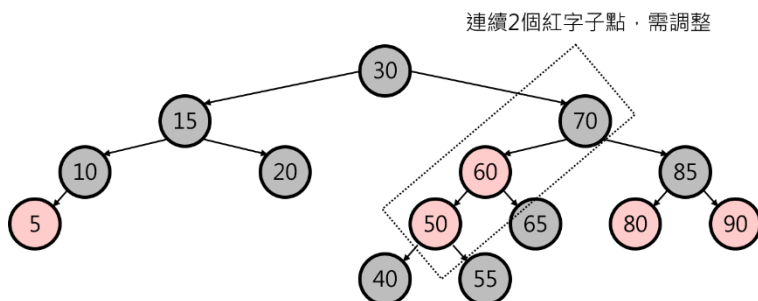
例 1：Insert 45, 2 之結果為何？



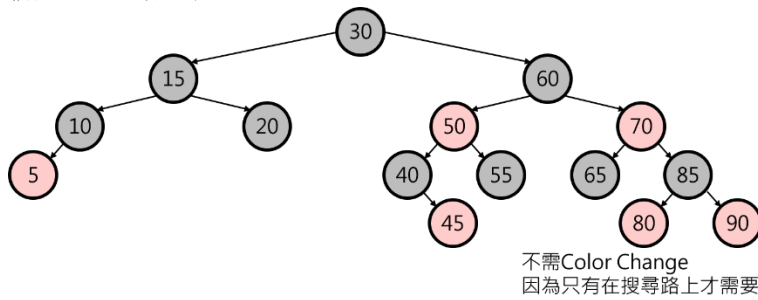
1. Insert 45，但在插入之前，因搜尋路上有 2 紅字點，故先 Color Change；



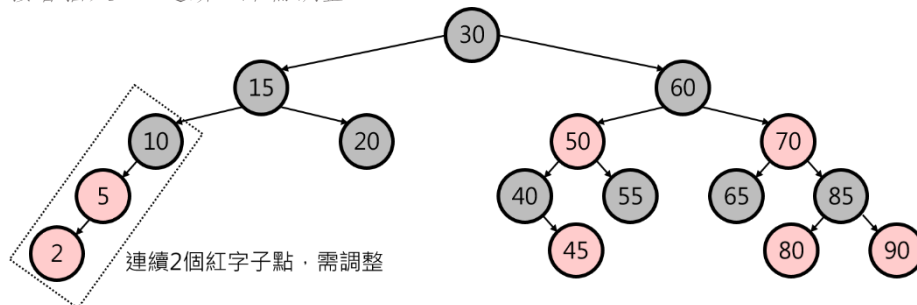
2. 之後有連續 2 紅，故調整



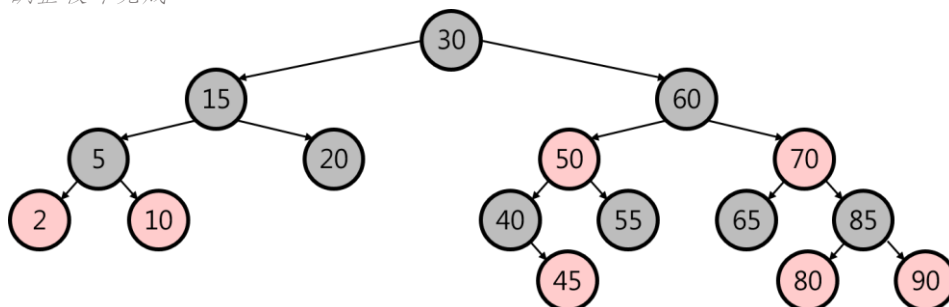
3. 插入 45 且為紅點



4. 接著插入 2，連續 2 點需調整

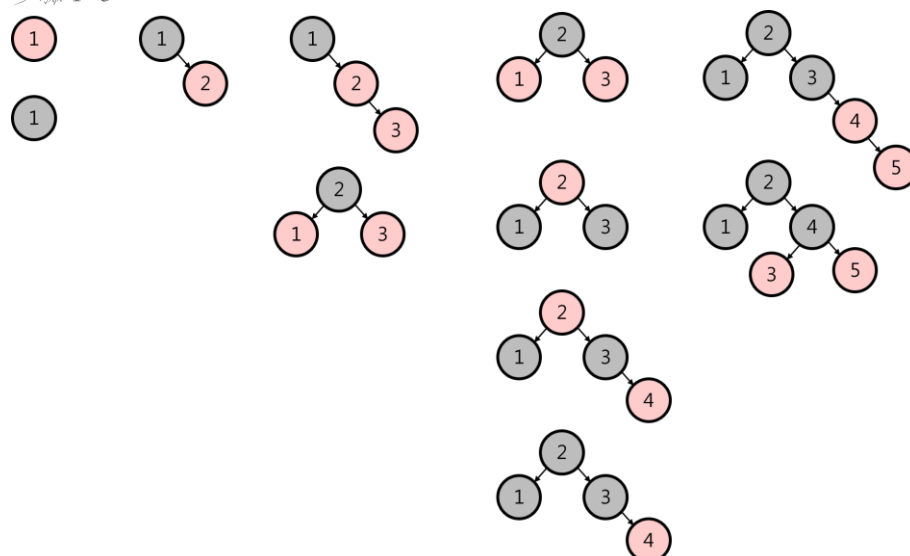


5. 調整後即完成

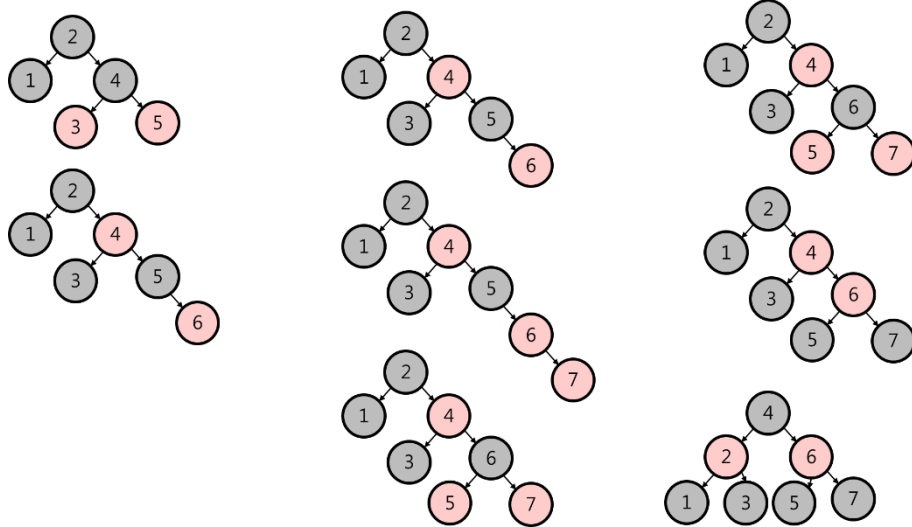


例 2：建一 RB Tree，依序為 1, 2, 3, 4, 5, 6, 7：

步驟 1~5：



步驟 5~7 :



Note :

沒有加權值，愈平衡愈好

⇒ ALV → RB Tree

有外部加權值

⇒ Huffman

內、外部皆有加權值

⇒ 採 OBST (Optimal)

Optimal Binary Search Tree(OBST)

Def :

給予 n 個內部加權值，令為 $p_i (1 \leq i \leq n)$

給予 $n+1$ 個外部加權值，令為 $q_j (0 \leq j \leq n)$

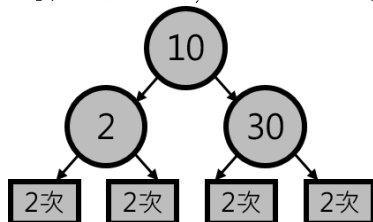
且 $\sum p_i + \sum q_j = 1$ ，在所有 BST 中，找具最小搜尋成本之 BST，謂之
搜尋成本 = 成功搜尋 + 失敗搜尋成本

成功搜尋成本：

$$\sum [(\text{內部 Node}_i \text{ 之 level 值}) * p_i]$$

失敗搜尋成本(本身不算)：

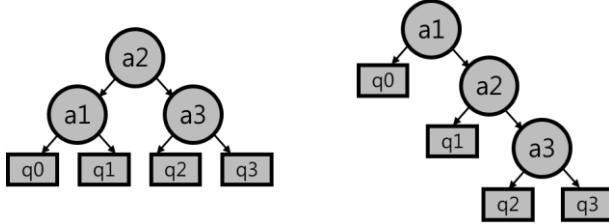
$$\sum [(\text{外部 Node}_j \text{ 之 level 值}-1) * q_j]$$



例：

Internal Node(a_1, a_2, a_3)加權值為(0.5, 0.1, 0.05)

External Node(q_0, q_1, q_2, q_3)加權為(0.15, 0.1, 0.05, 0.05)



1.

成功 $Cost = 1 \cdot 0.1 + 2 \cdot 0.5 + 2 \cdot 0.05 = 1.2$

失敗 $Cost = 2 \cdot 0.15 + 2 \cdot 0.1 + 2 \cdot 0.05 + 2 \cdot 0.05 = 0.7$

2.

成功 $Cost = 0.5 \cdot 1 + 0.15 \cdot 2 + 0.05 \cdot 3 = 0.85$

失敗 $Cost = 0.65$

由上可知：在此情況之下：平衡不見得最好

求 OBST 方法

[法一]採”Brute Force 暴力法”方法

將所有可能的 BST，成本一一求出，取最小 $\Rightarrow O(2^n)$

[法二]使用”Dynamic Programming”

1. 定義問題解法的運算表示式

2. 用表格逐步求解

3. 過程中會用到先前計算之結果

$\Rightarrow$ 最終求得”Optimal Solution”

[補充]Note：採 Dynamic Programming 可得最佳解的：

1. 0/1 背包問題
2. LCS 問題
3. 矩陣鏈相乘
4. One to all Shortest Path 不允許負邊(Bellman-Ford)
5. All to all Shortest Path 不允許負環(Floyd-Warshall)
6. OBST

求 OBST 作法

1. 令內部 Node 為 $a_{i+1}, a_{i+2}, \dots, a_j$

令 $T_{i,j}$ 為上述 Node 形成之 OBST

Note：

(1) $a_{i+1} \leq a_{i+2} \leq \dots \leq a_j$

(2) 內部加權值 $p_m, i+1 \leq m \leq j$

(3) 外部加樹值 $q_s, i \leq s \leq j$

例：

- (1) $T_{2,5}$ 表示由 a_3, a_4, a_5 組成的 OBST
- (2) $T_{4,6}$ 表示由 a_5, a_6 組成的 OBST
- (3) a_3, a_4, a_5 組成的 OBST 為 $T_{2,5}$
- (4) $a_1, a_2, \dots, a_n$ 組成的 OBST 為 $T_{0,n}$

2. 令 c_{ij} 為 T_{ij} 之 Cost

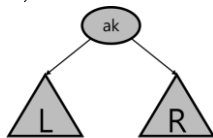
w_{ij} 為 T_{ij} 之內、外部加權值

r_{ij} 為 T_{ij} 之 Root

Note：

- (1) $T_{i,i} \Rightarrow$ 空樹， $c_{i,i}=0$ ， $r_{i,i}=0$
- (2) $T_{i,j}$ 不為 $i > j$ ，而定義。例： $T_{5,3}$ 不合法

3. c_{ij} 的推導公式：取任一點 a_k 作 Root



所以 L 由 $a_{i+1}, a_{i+2}, \dots, a_{k-1}$ 組成：L 的最佳為 $T_{i,k-1}$

R 由 $a_{k+1}, a_{k+2}, \dots, a_j$ 組成：R 的最佳為 $T_{k,j}$

可知： $c_{i,j} = 1 * P_k + c_{i,k-1} + c_{k,j} + w_{i,k-1} + w_{k,j}$ (L, R 因 Root 各多了一層)

$= w_{i,j} + c_{i,k-1} + c_{k,j}$

故 $w_{i,j} + \min\{c_{i,l-1} + c_{l,j}\}$ ，其中 $k=i+1 \leq l \leq j$

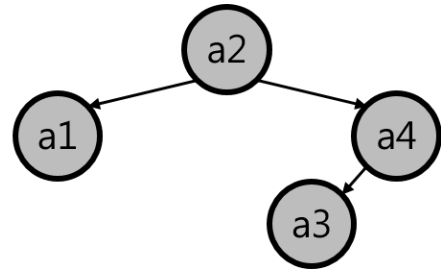
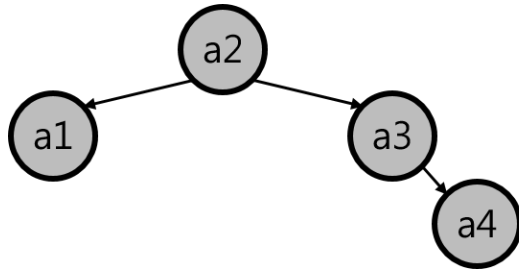
例：4 個內部 Node(a_1, a_2, a_3, a_4) 內部加權值： $(3, 3, 1, 1)$ 、外部加權： $(2, 3, 1, 1, 1)$

1. 求 OBST 之成本
2. 繪出 OBST 之結果

1.(相加方向口訣：頂向下、斜右上)

$w_{00}=2$ $c_{00}=0$ $r_{00}=null$	$w_{11}=3$ $c_{11}=0$ $r_{11}=null$	$w_{22}=1$ $c_{22}=0$ $r_{22}=null$	$w_{33}=1$ $c_{33}=0$ $r_{33}=null$	$w_{44}=1$ $c_{44}=0$ $r_{44}=null$
$w_{01}=8$ $c_{01}=8$ $r_{01}=1$	$w_{12}=7$ $c_{12}=7$ $r_{12}=2$	$w_{23}=3$ $c_{23}=3$ $r_{23}=3$	$w_{34}=3$ $c_{34}=3$ $r_{34}=4$	
$w_{02}=12$ $c_{02}=19$ $r_{02}=1$	$w_{13}=9$ $c_{13}=12$ $r_{13}=2$	$w_{24}=5$ $c_{24}=8$ $r_{24}=3 \text{ or } 4$		
$w_{03}=14$ $c_{03}=25$ $r_{03}=2$	$w_{14}=11$ $c_{14}=19$ $r_{14}=2$			
$w_{04}=16$ $c_{04}=32$ $r_{04}=2$				

2. 此 OBST 為：



Note：不見得唯一

Splay Tree(外張樹)

緣由：

	AVL Tree	Splay Tree
調整	複雜	簡單
實際成本	$O(\log n)$	$O(1) \sim O(n)$
分攤成本	-	$O(\log n)$

Def：Splay Tree 是一個 BST，Search, Insert, Delete x 之操作與 BST 相同，但多了一個”Splay” operation。令起點：

1. x, when search x, insert x
2. x 的父點, when delete x

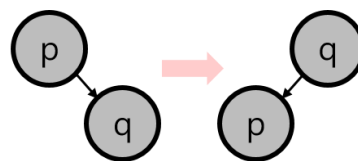
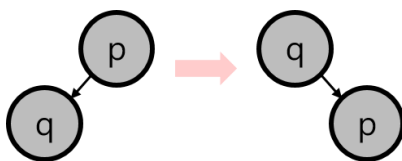
Note：

Splay：為一連串的 Rotation 組成

目的：將 Splay 起點逐步變成 Root

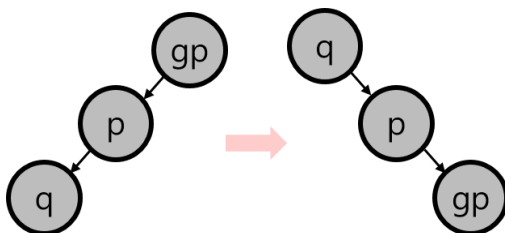
Rotation 之作法：

1. 2 個 Node

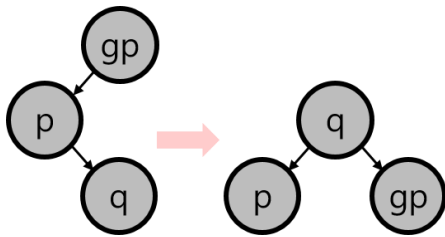


2. 3 個 Node：分為 LL、LR、RL、RR 四種(調整不同於 AVL Tree)

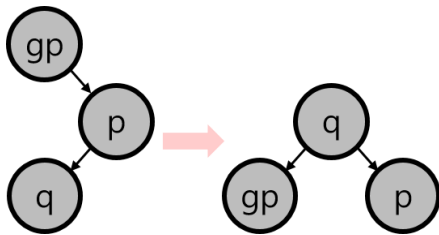
LL：



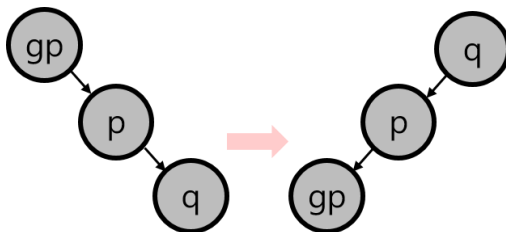
LR :



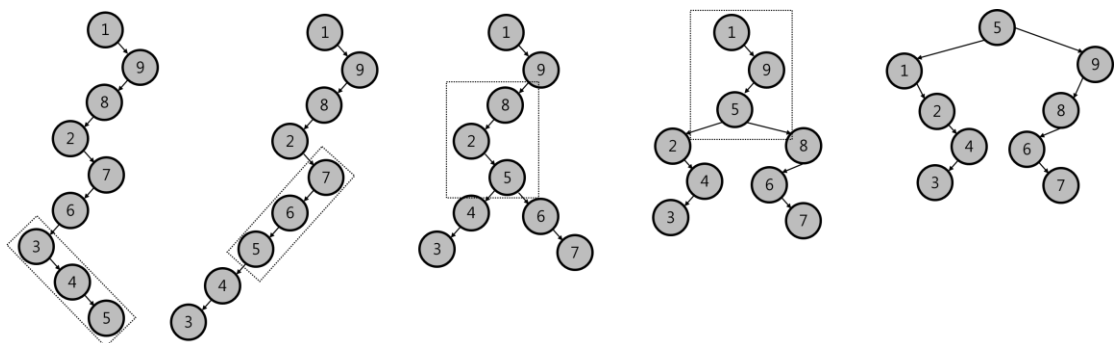
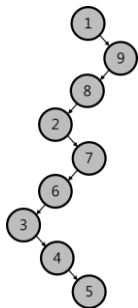
RL :



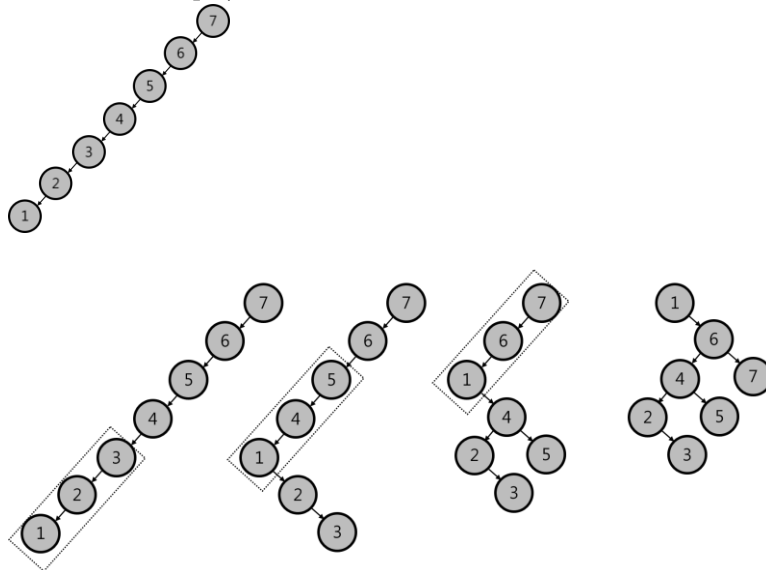
RR :



例 1：從 5 做 Splay，結果為何？



例 2：從 1 做 Splay，結果為何？



Leftist Tree(左傾樹)

提供 Double-Ended Priority Queue 的運作，以及 Combine 動作皆為 $O(\log n)$

Operation	Deap/Min-Max Heap	Leftist Tree
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$
Combine two Queue	$O(n)$	$O(\log n)$

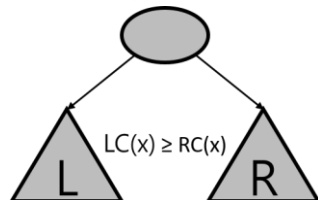
Shortest(x)：

Shortest(x) 為 x 到外部節點的最短路徑長度，令 x 為 Extended BT 的 Node，令 Leftchild(x)，Rightchild(x) 為 x 的左右子點，則：

$$\text{Shortest}(x) = 0, \quad \text{if } x \text{ is an external node}$$

$$1 + \min\{\text{Shortest}(\text{LC}(x)), \text{Shortest}(\text{RC}(x))\}$$

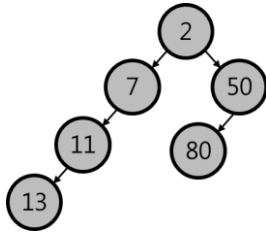
Def：若不為空，則每內部節點 x ，其 $\text{Shortest}(\text{LC}(x)) \geq \text{Shortest}(\text{RC}(x))$



Min Leftist Tree(最小左傾樹)

Def：為一 Leftist Tree，且其中各點之鍵值 $\leq$ 其子點的鍵值，反之為 Max Leftist Tree

例：



是最小左傾樹

相關運作：Insert, Delete, Combine, ... 等，皆為 $O(\log n)$

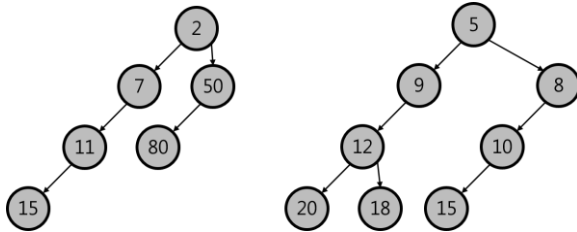
Note：Insert, Delete：分採用 Find & Union

Insert x 於 Leftist Tree：

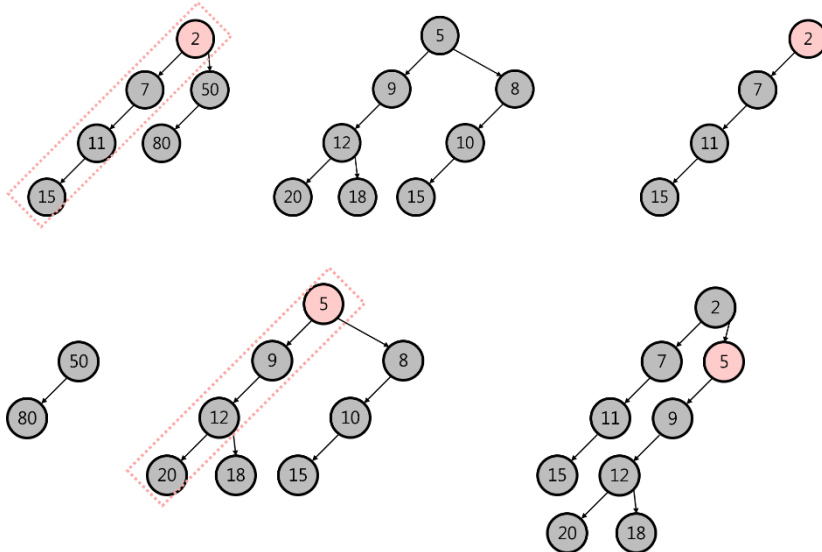
步驟：若樹為 A

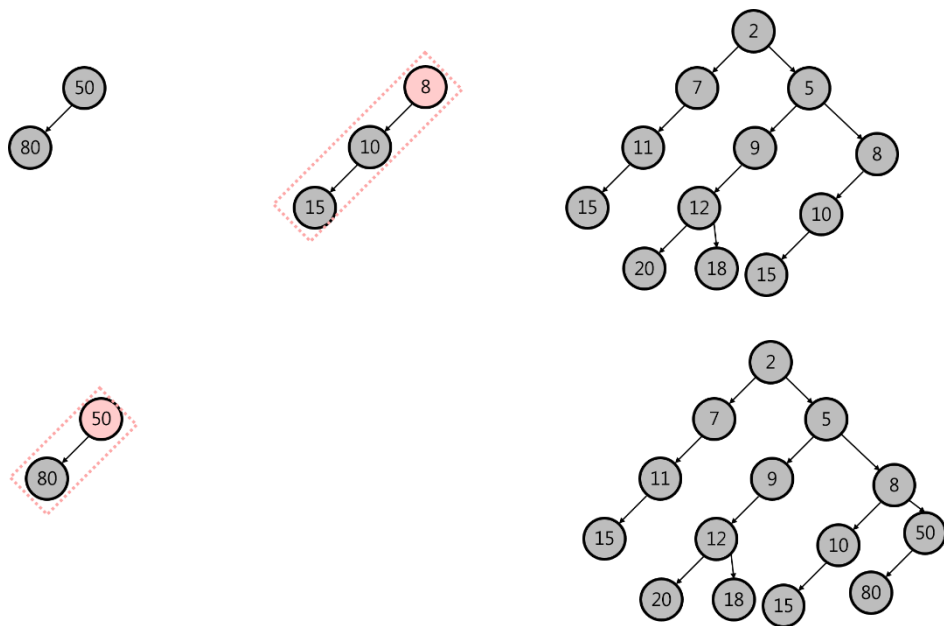
1. 建立一個只包含 x 的 Min-Leftist Tree B
2. 結合(Union) 2 個樹(A, B)
 - (1) 找出 $\min\{H_1.\text{Root}, H_2.\text{Root}\}$ 作為新樹的 Root；令 H_1 之 Root 有較小值
 - (2) H_1 之 Root 為 new Root，將 H_1 之左子樹保留
 - (3) Merge(H_1 右子樹, H_2)
 - (4) 最後形成一棵 Tree，check 是否滿足 Leftist Tree 之性質，若否則 Swap(左，右子樹)

例 1：問合併 H_1, H_2 之結果？(Leftist Tree)

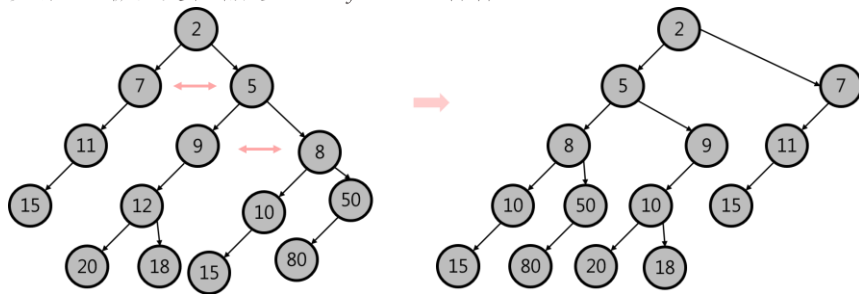


步驟 1：取最小值並保留最左子樹



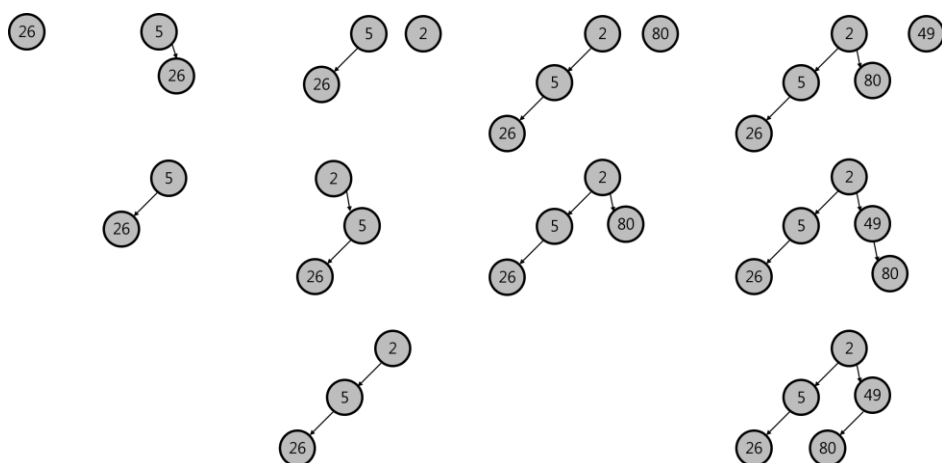


步驟 2：檢查是否滿足 Min Leftist Tree 條件

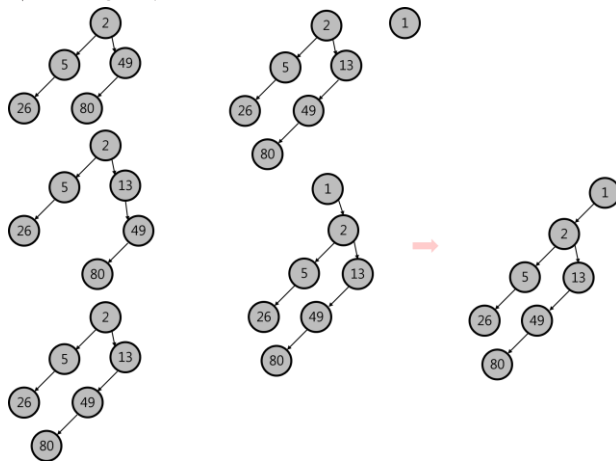


例 2：給 Data：26, 5, 2, 80, 49, 13, 1，建立 Min-Leftist Tree 結果為何？

第 1~5 步驟：



第 6~7 步驟：

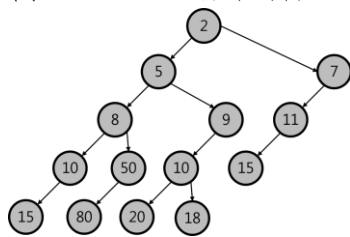


Delete Min 之運作

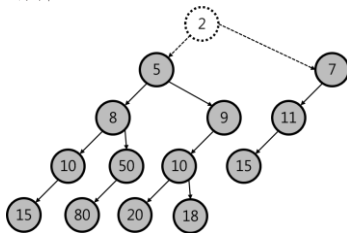
步驟：

1. Root 為最小值，將之 Delete，則左子樹(H_1)，右子樹(H_2)
2. 對 H_1 、 H_2 做 Merge

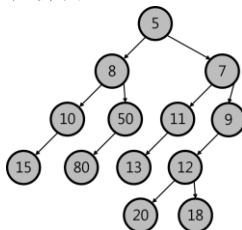
例：Delete Min 結果為何？



1. 刪掉 Root



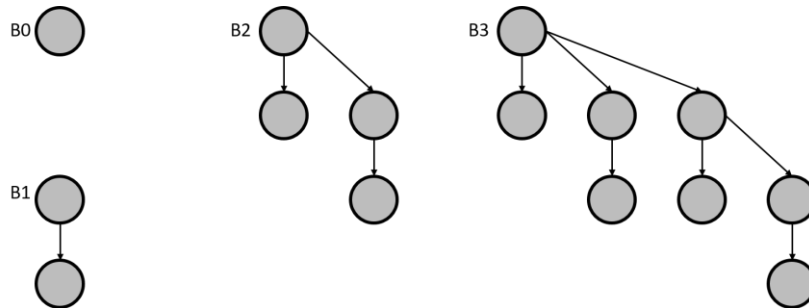
2. 合併兩 Tree



Binomial Tree

Def: 設 Root 在 level 0, 令 B_0 是高度 0 的 Binomial Tree, 且只有一個點, 而高度為 k 的 Binomial Tree B_k 是由 2 個 B_{k-1} 組成
任取一棵 B_{k-1} 之 Root 為 new Root, 另一棵 B_{k-1} 為其子樹

例:



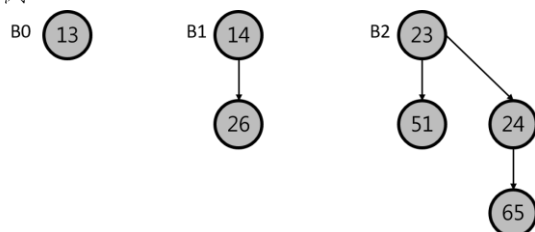
Note: 相關數學式:

1. 高度 k 的 B_k 有 2^k 個 Nodes
2. 高度 k 中, level i 之 Node 為 C_i^k 個

Binomial Heap

Def: 由 ≥ 0 棵 Binomial Tree 組成之 Forest(或集合), 且各棵 Binomial Tree 亦滿足 Min-Tree 性質, 指 $\text{Parent} \leq \text{Child}$ 值

例:



Note:

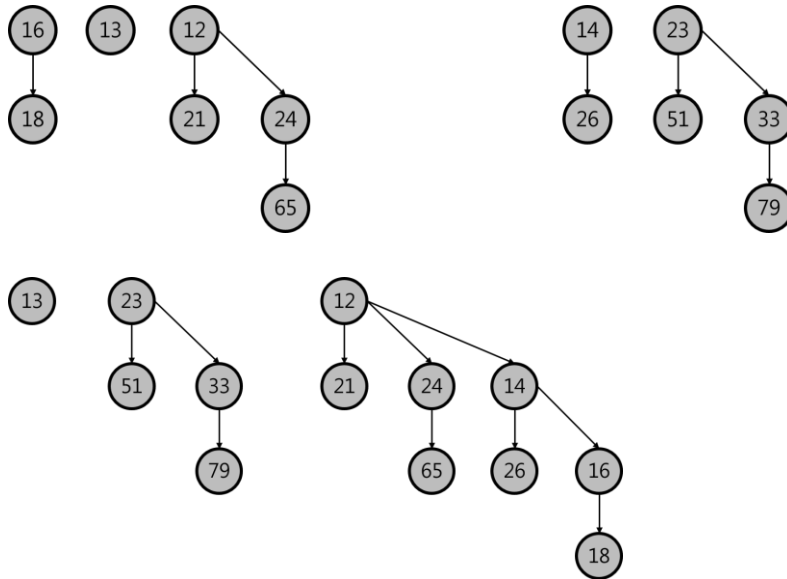
1. 7 個元素之 Binomial Heap 由 B_0, B_1, B_2 組成, 因為 $7_{10} = 111_2$
2. 13 個元素: $13_{10} = 1101_2$
3. n 個元素: 需 $\log n$ 個 bits, 需 $O(\log n)$ 棵 Binomial Tree 組成

Insert

令欲 Merge(H_1, H_2)

1. 具相同高度的 Binomial Tree 合併
2. 2 棵 Tree 合併, 以最小 Root 為 new Root, 另一棵為其子樹
3. Repeat 1. 與 2. 直到沒有相同高度可合併, 停止, 若有 >2 棵高度相同, 則任選 2 個 Tree 合併(一般選最小 2 棵)

例：



Note：答案不見得唯一

分析：

大部分的情況之下，Insert x 需花 $O(1)$

少部分的情況之下，Insert x 需花 $O(\log n)$

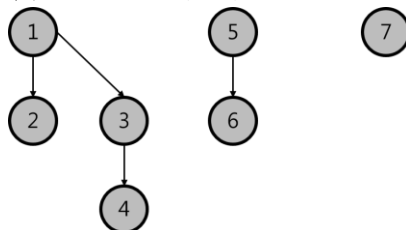
分攤成本： $O(1)$

Delete Min in Binomial Heap

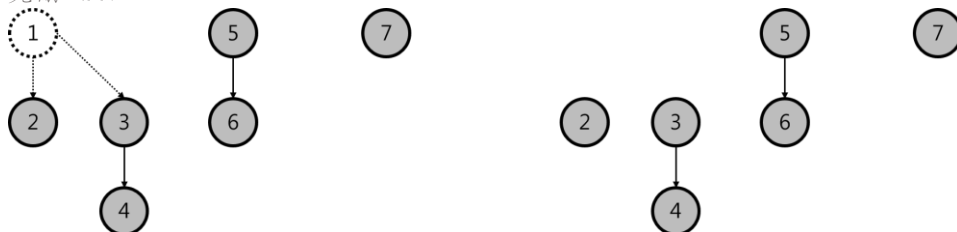
步驟：

1. 找出最小 Root 之 Binomial Tree： T ，令其他集合為 H_1
2. 將 T 之 Root Delete，它的子樹集合為 H_2
3. Merge (H_1, H_2)即可

例：Delete 結果為何？



1. 先刪 Root



2. 再合併：



Fibonacci Heap(費式堆積)

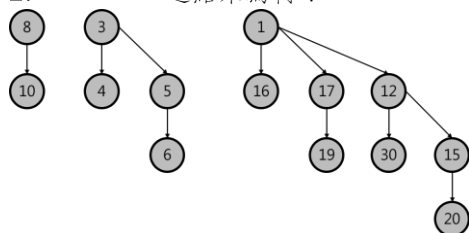
Def：是 Extended Binomial Heap，除提供 Insert, Delete, Merge 之外，另加了 2 項運作：

1. Delete a Node(key)：Delete 一特定值
2. Decrease the key of a Node：對一 Node 做”減”的動作

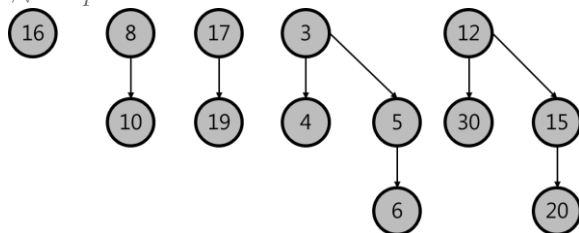
Note：採”Lazy Merge(延遲合併)”：指具相同高之 Binomial Tree 未合併，暫時不處理

例 1：

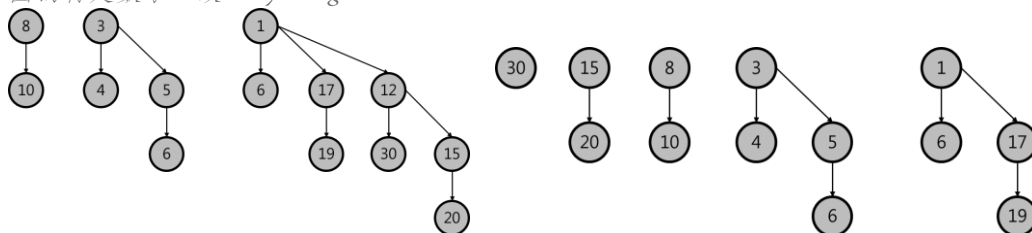
1. Delete 1 之結果為何？
2. Delete 12 之結果為何？



1. 同 Heap

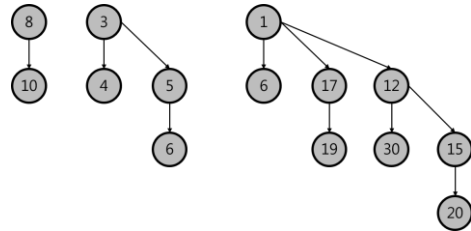


2. 因為特定數字，故 Lazy Merge：



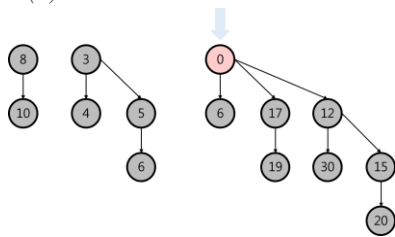
例 2：以下四題各自獨立

1. 對 1 減 1
2. 對 3 減 3
3. 對 15 減 2
4. 對 15 減 4

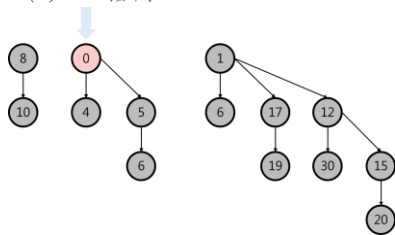


指標指向最小值

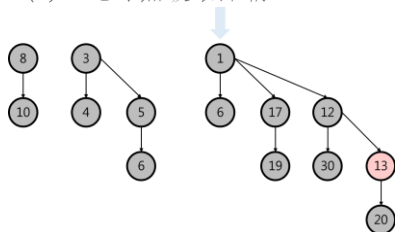
1. $O(1)$



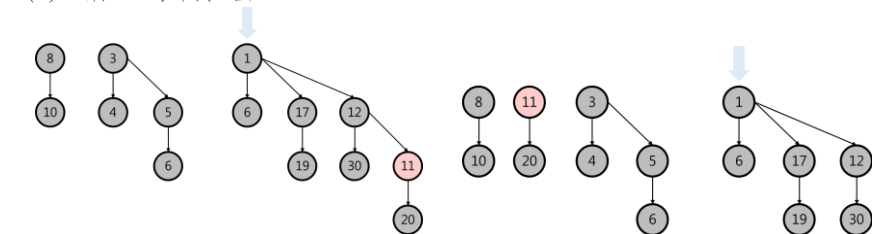
2. $O(1)$, a 指向 0



3. $O(1)$, 此時無破壞結構



4. $O(1)$, 將 11 子樹拉出



[Thomas C 演算法版]

Operation	Binary Heap(Worst, DS 特愛考)	Binomial Heap(Worst)	Fibonacci Heap(攤)
1. Make-Heap	$O(1)$	$O(1)$	$O(1)$
2. Insert	$O(\log n)$	$[1]O(\log n)$	$O(1)$
3. Find-Min	$O(1)$	$[2]O(\log n)$	$[3]O(1)$
4. Extract-Min	$O(\log n)$	$O(\log n)$	$O(\log n)$
5. Union	$O(n)$	$O(\log n)$	$[4]O(1)$
6. Decrease key	$O(\log n)$	$[5]O(\log n)$	$O(1)$
7. Delete	$[6]O(\log n)$	$O(\log n)$	$O(\log n)$

[上表之補充說明]

[1]Binomial Heap 的 Worst 下，Insert $O(\log n)$ ，但分攤下為 $O(1)$

[2]因為 Binomial Heap 最差下需比較 $\log n$ 棵樹的 Root 找 min

[3]Fibonacci Heap 有設定指標指向 Min

[4]因為 Fibonacci Heap 採用 Lazy Merge

[5]例：看高度，因此 $O(\log n)$

[6]用最差情況 Delete Min 來考量： $O(\log n)$

